

Министерство образования Московской области
Государственное бюджетное профессиональное образовательное
учреждение
Московской области
«Сергиево-Посадский колледж»

Специальность 09.02.07
Информационные системы и программирование

ДИПЛОМНЫЙ ПРОЕКТ

Тема: Разработка веб приложения для мониторинга и управления
рабочей деятельностью сотрудников ООО “Интернет-Эксперт”

Студент: Федин Максим Алексеевич

Руководитель проекта: Карцева Мария Сергеевна

« » июня 2026 г. Подпись

Консультант по организационно-экономической части: Веденкина Ирина Владимировна

« » июня 2026 г. Подпись

Нормоконтроль: Данилина Мария Владимировна

« » июня 2026г. Подпись _____

Рецензент: Недобоев Евгений Евгеньевич

« » июня 2026 г. Подпись

г. Сергиев Посад
2026

Содержание

Введение	4
1 Анализ предметной области	7
1.1 Сведения об организации	7
1.2 Информационная и функциональная структура	9
1.3 Классификация информационных систем	13
1.4 Постановка задачи	18
1.4.1 Аналогии информационной системы	20
1.5 Обоснование выбора ПО для реализации проекта	23
2 Проектная часть	28
2.1 Проектирование архитектуры проекта	28
2.1.1 Проектирование базы данных	39
2.2 Разработка интерфейсов	48
2.3 Разработка клиентской части	53
2.3.1 Руководство пользователя	72
2.4 Разработка серверной части	76
2.5 Тестирование информационной системы	89
3 Экономическое обоснование	97
3.1 Определение затрат на проектирование	97
3.2 Расчет экономической эффективности	101
Заключение	104
Библиография	107
Приложение А	110
Приложение Б	115

Введение

В условиях цифровизации и роста объемов обрабатываемой информации в современных организациях возрастает потребность в использовании специализированных программных решений, обеспечивающих эффективное управление проектами, задачами и взаимодействием сотрудников. Использование разрозненных инструментов, таких как электронная почта, электронные таблицы и локальные документы, приводит к снижению эффективности работы, усложняет контроль выполнения задач и затрудняет организацию совместной деятельности.

Одним из способов решения данной проблемы является внедрение централизованной информационной системы, позволяющей объединить процессы управления проектами, распределения задач и взаимодействия участников в едином программном продукте. Подобный подход обеспечивает повышение прозрачности рабочих процессов, упрощает контроль выполнения задач и способствует более эффективному использованию ресурсов организации.

В рамках дипломного проекта разрабатывается программный продукт “Atlas”, предназначенный для организации командной работы, управления рабочими пространствами и контроля выполнения задач. Система реализована в формате кроссплатформенного desktop-приложения и может использоваться на операционных системах Windows 11, Linux и macOS.

Архитектура программного продукта построена по клиент-серверному принципу. Серверная часть разработана на языке программирования Go с использованием системы управления базами данных PostgreSQL. Для обмена данными между клиентом и сервером используется протокол gRPC с поддержкой потоковой передачи данных, обеспечивающий возможность синхронизации информации в режиме реального времени. Клиентская часть реализована с использованием фреймворка Wails и библиотеки React, что

позволяет создать современный пользовательский интерфейс и обеспечить работу приложения на различных операционных системах.

Функциональные возможности системы включают регистрацию и авторизацию пользователей с использованием электронной почты и пароля, а также поддержку внешних провайдеров аутентификации. После регистрации каждому пользователю автоматически создается персональное рабочее пространство, предназначенное для организации индивидуальной работы.

Система предоставляет возможность создания командных рабочих пространств, приглашения участников и управления их ролями. Для каждого рабочего пространства реализован механизм разграничения прав доступа, предусматривающий роли владельца, администратора и участника. Это позволяет организовать безопасную совместную работу пользователей и обеспечить контроль над действиями участников проекта.

Внутри рабочих пространств реализована возможность создания kanban-досок, предназначенных для визуального управления задачами. Каждая доска содержит набор колонок и карточек задач, позволяющих отслеживать текущее состояние работ. Карточки задач содержат информацию о наименовании задачи, приоритете и сроке выполнения, а также поддерживают прикрепление файлов, необходимых для выполнения работы.

Дополнительно система предоставляет средства управления учетной записью пользователя, включая изменение персональных данных, настройку параметров приложения, управление способами авторизации и выбор пользовательских предпочтений интерфейса.

Серверная часть приложения обеспечивает обработку запросов пользователей, реализацию бизнес-логики, контроль прав доступа, хранение данных и синхронизацию состояния между участниками проекта. Централизованный подход к обработке информации позволяет обеспечить целостность данных, безопасность работы системы и возможность ее дальнейшего развития.

Основными целями разработки программного продукта являются:

- автоматизация управления задачами и рабочими процессами;
- организация совместной работы пользователей в рамках рабочих пространств;
- централизованное хранение и обработка данных;
- обеспечение безопасной аутентификации и авторизации пользователей;
- реализация разграничения прав доступа между участниками проектов;
- повышение эффективности контроля выполнения задач;
- создание масштабируемой и расширяемой программной архитектуры.

Разработка программного продукта “Atlas” позволит повысить эффективность организации рабочих процессов, упростить взаимодействие между участниками проектов и обеспечить удобный инструмент для управления задачами и рабочими пространствами в рамках единой информационной системы.

1 Анализ предметной области

1.1 Сведения об организации

Компания ООО “Интернет-Эксперт” осуществляет деятельность в сфере информационных технологий с 2007 года и специализируется на IT-аутсорсинге, разработке и сопровождении web-проектов, внедрении CRM-систем, автоматизации бизнес-процессов, а также аналитических решениях для корпоративных клиентов. Организация зарегистрирована в Московской области, г. Сергиев Посад, и за время своей работы реализовала более 350 проектов различного уровня сложности.

Основным направлением деятельности компании является разработка и сопровождение программных решений для автоматизации внутренних и внешних бизнес-процессов организаций. Компания сотрудничает с различными коммерческими структурами и предоставляет услуги по разработке программного обеспечения, техническому сопровождению, настройке серверной инфраструктуры и внедрению корпоративных систем.

В связи с увеличением количества внутренних проектов и расширением технической инфраструктуры компании было принято решение о создании нового тестового отдела разработки. Особенностью данного подразделения является использование языка программирования Go для разработки серверных приложений и внутренних сервисов компании. Создание отдельного подразделения обусловлено необходимостью тестирования новых технологий, повышения производительности внутренних сервисов и внедрения современных подходов к разработке программного обеспечения.

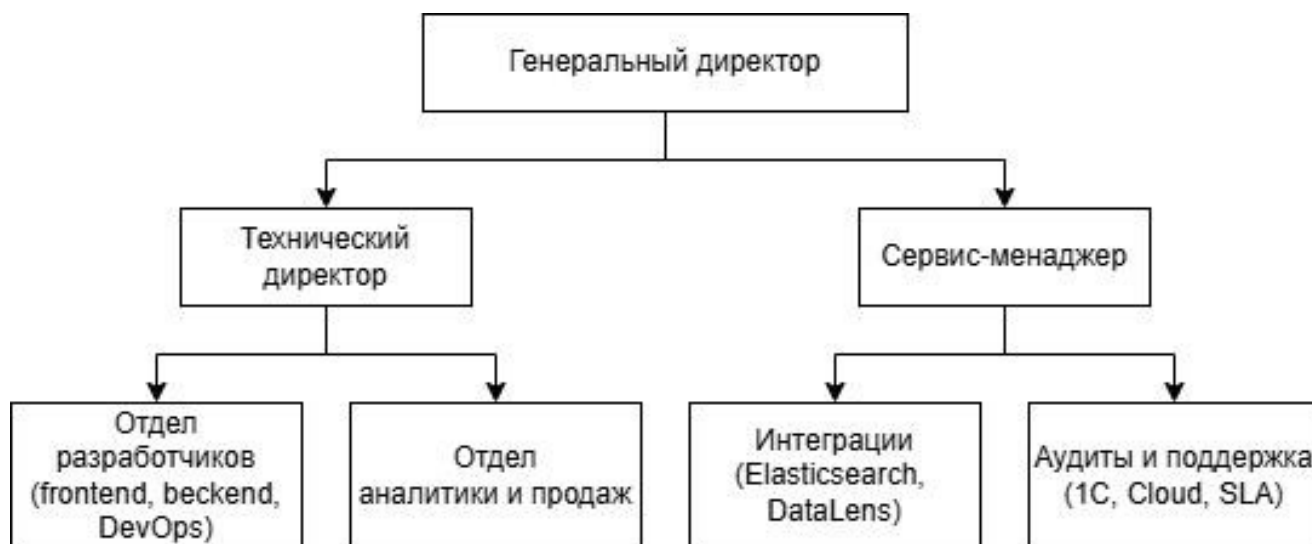


Рисунок 1.1 - Схема подчинения

Организационная структура организации компании (Рисунок 1.1) включает несколько основных подразделений:

- руководство компании, осуществляющее контроль рабочих процессов и распределение ресурсов;
- отдел продаж, отвечающий за взаимодействие с клиентами и сопровождение договоров;
- отдел разработки, занимающийся созданием и сопровождением программного обеспечения;
- отдел DevOps и инфраструктуры, обеспечивающий настройку серверов и поддержку сервисов;
- отдел аналитики, выполняющий подготовку отчетности и анализ данных;
- отдел технической поддержки, осуществляющий сопровождение пользователей и обработку обращений;
- тестовый отдел разработки на Go, предназначенный для создания и тестирования внутренних корпоративных сервисов.

В процессе работы нового подразделения возникла необходимость в использовании единой системы управления задачами и проектами. На момент

начала разработки сотрудники использовали сторонние сервисы и разрозненные инструменты для ведения задач, обмена информацией и контроля этапов выполнения работ. Такой подход усложнял взаимодействие между участниками команды, затруднял отслеживание изменений и снижал прозрачность рабочих процессов.

Дополнительной проблемой являлось отсутствие единого пространства для хранения информации о проектах, задачах и действиях пользователей. Использование различных сервисов приводило к потере части данных, дублированию информации и увеличению времени на выполнение организационных задач. В результате было принято решение о разработке собственной системы управления проектами, ориентированной на внутренние процессы компании и особенности работы тестового отдела разработки.

Разрабатываемое приложение предназначено для централизованного управления задачами, распределения ролей между сотрудниками, контроля выполнения работ и организации совместного взаимодействия внутри команды разработки. Использование собственной системы позволяет адаптировать функциональность под требования компании и обеспечить возможность дальнейшего масштабирования проекта.

1.2 Информационная и функциональная структура

Информационная структура проектируемой системы представляет собой централизованное пространство для взаимодействия сотрудников внутри тестового отдела разработки компании ООО “Интернет-Эксперт”. Основное назначение системы заключается в хранении, обработке, передаче и отображении информации о проектах, задачах, пользователях и этапах выполнения работ.

На текущий момент внутри компании используется устаревшая внутренняя система управления задачами, разработанная на языке программирования PHP. Данная система создавалась для решения базовых

задач организации рабочего процесса, однако со временем перестала соответствовать требованиям компании и современным подходам к разработке корпоративного программного обеспечения.

Существующая система имеет ряд недостатков:

- низкая производительность при большом количестве запросов;
- сложность масштабирования проекта;
- устаревшая архитектура приложения;
- ограниченные возможности обновления информации в режиме реального времени;
- неудобный пользовательский интерфейс;
- сложность поддержки и расширения функциональности;
- высокая нагрузка на серверную часть при одновременной работе нескольких пользователей.

Дополнительной проблемой является использование большого количества разрозненных инструментов для организации рабочих процессов. Часть информации хранится внутри старой системы, часть задач фиксируется в сторонних сервисах, а обмен информацией между сотрудниками осуществляется через мессенджеры и другие внешние платформы. Такой подход усложняет контроль выполнения задач и увеличивает вероятность потери информации.

В существующей схеме работы сотрудники создают задачи вручную, распределяют обязанности между участниками проекта и отслеживают этапы выполнения работ через отдельные сервисы. Руководители проектов контролируют выполнение задач посредством внутренней системы и сторонних инструментов, а информация о текущем состоянии проектов передается между сотрудниками вручную.

Из-за отсутствия централизованного хранения данных часть информации дублируется в нескольких сервисах одновременно. Это приводит

к увеличению времени обработки информации, усложняет взаимодействие между сотрудниками и снижает эффективность организации рабочих процессов внутри подразделения.

Для решения существующих проблем было принято решение о разработке новой информационной системы, ориентированной на внутренние процессы тестового отдела разработки и современные подходы к организации командной работы.

Информационная структура проектируемой системы включает несколько основных сущностей:

- рабочие пространства;
- рабочие пространства;
- доски задач;
- задачи и подзадачи;
- комментарии;
- вложения;
- роли пользователей;
- уведомления и события системы.

Рабочее пространство используется в качестве основной области взаимодействия сотрудников. Внутри рабочего пространства создаются проекты и доски задач, предназначенные для организации процессов разработки и распределения обязанностей между участниками команды.

Каждый проект содержит набор задач, связанных с определенным направлением разработки или этапом выполнения работ. Задачи могут содержать описание, исполнителя, статус выполнения, дату создания, комментарии и прикрепленные файлы. Использование комментариев позволяет участникам проекта обмениваться информацией и отслеживать историю изменений по каждой задаче.

Доски задач используются для визуального отображения этапов выполнения работ. Перемещение задач между колонками позволяет отображать текущее состояние разработки и упрощает контроль выполнения задач внутри команды.

Функциональная структура приложения предусматривает разделение системы на клиентскую и серверную части. Серверная часть разработана на языке программирования Go и отвечает за обработку запросов, хранение информации, управление пользователями и синхронизацию данных между участниками системы.

Использование языка Go позволяет обеспечить высокую производительность серверной части и стабильную работу системы при одновременном подключении нескольких пользователей. Серверная часть реализует обработку бизнес-логики приложения, взаимодействие с базой данных и передачу информации между компонентами системы.

Взаимодействие между клиентской и серверной частью реализовано с использованием технологии gRPC. Использование данного подхода обеспечивает высокую скорость обмена данными, упрощает организацию удаленного взаимодействия между сервисами и позволяет снизить нагрузку на серверную часть приложения.

Для передачи обновлений в режиме реального времени используются gRPC Stream-соединения. Данный механизм позволяет серверной части автоматически передавать изменения клиенту без необходимости постоянной отправки повторных запросов. Благодаря этому пользователи могут оперативно получать обновления по задачам, комментариям и действиям других участников проекта.

В качестве клиентской части используется desktop-приложение, разработанное с использованием фреймворка Wails. Использование desktop-приложения позволяет обеспечить более стабильную работу системы, повысить производительность интерфейса и предоставить пользователям

единое программное решение для взаимодействия с корпоративной системой управления задачами.

Информационная система обеспечивает следующие возможности:

- централизованное хранение информации о проектах и задачах;
- управление ролями пользователей и правами доступа;
- отслеживание этапов выполнения задач;
- взаимодействие между участниками команды;
- хранение истории изменений;
- контроль выполнения рабочих процессов;
- распределение обязанностей между сотрудниками;
- управление проектами внутри рабочего пространства.

Разрабатываемое приложение ориентировано на оптимизацию процессов командной разработки и повышение эффективности взаимодействия сотрудников внутри подразделения. Использование единой платформы позволяет централизовать хранение информации, сократить время обработки данных, повысить прозрачность рабочих процессов и упростить контроль выполнения задач внутри команды разработки.

1.3 Классификация информационных систем

Информационная система представляет собой программно-технический комплекс, предназначенный для хранения, обработки, передачи и отображения информации, необходимой для выполнения определенных задач внутри организации. В настоящее время информационные системы применяются практически во всех сферах деятельности и позволяют автоматизировать различные бизнес-процессы, связанные с обработкой данных и организацией взаимодействия между пользователями.

Существует несколько основных классов информационных систем, различающихся по назначению, структуре, способу обработки информации и области применения.

По уровню управления информационные системы подразделяются на:

- системы оперативного уровня;
- системы уровня специалистов;
- системы уровня управления;
- стратегические информационные системы.

Системы оперативного уровня используются для выполнения повседневных операций организации и обработки большого количества однотипной информации. Основной задачей таких систем является автоматизация рутинных процессов и обеспечение стабильной работы организации.

Системы уровня специалистов предназначены для обработки специализированной информации и используются сотрудниками определенных подразделений для выполнения профессиональных задач. К таким системам относятся системы проектирования, аналитические платформы и средства управления разработкой программного обеспечения.

Системы уровня управления применяются для контроля деятельности организации, анализа показателей и принятия управленческих решений. Они обеспечивают сбор, обработку и отображение информации, необходимой руководству компании.

Стратегические информационные системы используются для долгосрочного планирования и анализа развития организации. Подобные решения позволяют прогнозировать изменения, анализировать эффективность деятельности и формировать стратегию развития компании.

По функциональному назначению информационные системы подразделяются на:

- системы управления предприятием;
- системы управления проектами;
- системы документооборота;
- аналитические системы;
- справочные системы;
- информационно-поисковые системы;
- системы поддержки принятия решений;
- корпоративные информационные системы;
- CRM-системы;
- ERP-системы.

Системы управления проектами предназначены для организации командной работы, распределения задач между сотрудниками и контроля выполнения этапов работ. Основной особенностью подобных систем является наличие инструментов для взаимодействия между участниками проекта, отслеживания изменений и управления рабочими процессами.

Корпоративные информационные системы используются для автоматизации внутренних процессов организации и обеспечения централизованного хранения информации. Такие системы объединяют различные подразделения компании в единое информационное пространство и обеспечивают взаимодействие между сотрудниками.

CRM-системы предназначены для взаимодействия с клиентами и автоматизации процессов продаж. ERP-системы используются для комплексного управления ресурсами предприятия и объединяют большое количество внутренних бизнес-процессов организации.

По способу организации архитектуры информационные системы подразделяются на:

- локальные;
- файл-серверные;
- клиент-серверные;
- распределенные;
- облачные.

Локальные системы функционируют на одном устройстве и не предусматривают сетевого взаимодействия между пользователями. Файл-серверные системы обеспечивают совместный доступ к данным через файловое хранилище, однако имеют ограничения по производительности и безопасности.

Клиент-серверные системы используют разделение приложения на клиентскую и серверную части. Серверная часть отвечает за обработку запросов и хранение информации, а клиентская обеспечивает взаимодействие пользователя с системой. Такой подход позволяет повысить производительность, безопасность и масштабируемость приложения.

Распределенные системы состоят из нескольких взаимодействующих сервисов и обеспечивают обработку данных между различными узлами сети. Облачные системы размещаются на удаленной инфраструктуре и предоставляют доступ к функциональности через сеть Интернет.

По характеру обработки данных информационные системы подразделяются на:

- однопользовательские;
- многопользовательские;
- системы пакетной обработки;
- системы реального времени.

Однопользовательские системы предназначены для работы одного пользователя и не требуют синхронизации данных между несколькими участниками. Многопользовательские системы обеспечивают одновременную работу нескольких пользователей и требуют постоянного обновления информации между клиентами системы.

Системы реального времени позволяют моментально передавать изменения между участниками и обеспечивают актуальность отображаемых данных. Подобный подход особенно важен для систем управления проектами и командной работы, где пользователи должны оперативно получать информацию об изменениях задач и действий других сотрудников.

Разрабатываемое приложение относится к классу корпоративных информационных систем управления проектами и задачами. Основное назначение системы заключается в организации совместной работы сотрудников, централизованном управлении проектами и контроле выполнения задач внутри тестового отдела разработки компании.

По функциональному назначению проектируемая система наиболее близка к классу систем управления проектами и корпоративных информационных систем, так как приложение обеспечивает организацию взаимодействия между сотрудниками, распределение задач, управление ролями пользователей и хранение информации о проектах.

По типу архитектуры система относится к клиент-серверным desktop-приложениям. Пользователь взаимодействует с системой через установленное desktop-приложение, подключающееся к серверной части посредством сетевого взаимодействия.

По характеру обработки данных проектируемая система относится к многопользовательским информационным системам с поддержкой работы в режиме реального времени. Использование потоковой передачи данных через gRPC Stream позволяет обеспечивать постоянную синхронизацию информации между пользователями системы и оперативно отображать изменения задач, комментариев и действий участников проекта.

Таким образом, поставленная задача наиболее соответствует классу корпоративных информационных систем управления проектами и командной разработкой, так как разрабатываемое приложение предназначено для организации совместной работы сотрудников, централизованного хранения информации и управления внутренними процессами подразделения компании.

1.4 Постановка задачи

В процессе работы тестового отдела разработки компании ООО “Интернет-Эксперт” возникает необходимость в организации эффективного взаимодействия между сотрудниками, централизованном управлении проектами и контроле выполнения задач внутри команды. Использование устаревших внутренних решений и сторонних сервисов не позволяет полностью адаптировать рабочие процессы под требования компании, а также создает сложности при масштабировании системы и внедрении новой функциональности.

Основной задачей разработки является создание корпоративного desktop-приложения “Atlas”, предназначенного для организации командной работы, управления проектами и взаимодействия сотрудников внутри рабочих пространств.

Разрабатываемая система должна обеспечивать централизованное хранение информации о пользователях, проектах, задачах и действиях участников системы. Дополнительно приложение должно обеспечивать обмен данными между клиентской и серверной частью в режиме реального времени, а также предоставлять удобный пользовательский интерфейс для взаимодействия сотрудников внутри команды.

Приложение предназначено для эксплуатации внутри компании ООО “Интернет-Эксперт” и ориентировано на использование тестовым отделом разработки при организации внутренних рабочих процессов.

В рамках разработки программного продукта необходимо реализовать:

- систему регистрации и авторизации пользователей;
- поддержку авторизации через сторонних провайдеров;
- создание и управление рабочими пространствами;
- систему ролей и разграничения прав доступа;
- управление участниками рабочего пространства;
- создание, редактирование и удаление задач;
- kanban-доску для отображения этапов выполнения задач;
- drag-and-drop для перемещения задач между колонками;
- систему комментариев и взаимодействия между пользователями;
- поддержку обновления данных в режиме реального времени;
- управление пользовательскими настройками приложения;
- поддержку смены темы интерфейса и языка системы;
- управление профилем пользователя и персональными данными.

После запуска программы пользователю должна отображаться форма входа в учетную запись. Система должна поддерживать регистрацию новых пользователей, а также возможность авторизации через сторонние сервисы.

После успешной авторизации пользователя необходимо обеспечить переход в основную рабочую область приложения (dashboard), предназначенную для дальнейшего взаимодействия с системой.

В интерфейсе системы необходимо реализовать следующие основные разделы:

- Рабочие пространства - для создания, просмотра и управления рабочими пространствами пользователя, а также взаимодействия с участниками и приглашениями;
- Доски - для работы с kanban-досками и задачами в рамках выбранного рабочего пространства;

- Настройки - для управления параметрами приложения, включая настройку темы оформления, языка интерфейса и выполнение выхода из учетной записи;
- Профиль - для отображения и редактирования персональных данных пользователя, управления учетными данными и параметрами авторизации.

Реализация указанных разделов должна обеспечить удобную навигацию по системе, доступ к основным функциям приложения и эффективное взаимодействие пользователей с рабочими пространствами и задачами. Для взаимодействия между клиентской и серверной частью системы используется технология gRPC. Обновление информации в режиме реального времени реализовано с использованием gRPC Stream-соединений, позволяющих передавать изменения между участниками системы без необходимости повторной отправки запросов.

В качестве клиентской части используется desktop-приложение, разработанное с использованием фреймворка Wails. Серверная часть реализована на языке программирования Go.

Разрабатываемое приложение должно обеспечивать стабильную работу системы при одновременном подключении нескольких пользователей, возможность дальнейшего расширения функциональности и поддержку масштабирования архитектуры приложения в процессе развития проекта.

1.4.1 Аналоги информационной системы

На сегодняшний день существует большое количество информационных систем, предназначенных для управления проектами, организации командной работы и контроля выполнения задач. Подобные решения используются как небольшими командами разработки, так и крупными организациями для распределения обязанностей между

сотрудниками, отслеживания этапов выполнения работ и централизованного хранения информации о проектах.

К наиболее распространенным системам данного типа относятся YouGile, Trello, а также другие сервисы для организации совместной работы сотрудников.

Система YouGile представляет собой платформу для управления проектами и задачами с использованием досок, колонок и карточек задач. Основное внимание в данной системе уделяется визуальному отображению этапов выполнения задач и организации взаимодействия между участниками команды. Пользователи могут создавать проекты, назначать исполнителей, изменять статусы задач и обмениваться комментариями внутри рабочего пространства.

Модель организации интерфейса и логика взаимодействия пользователей в разрабатываемом приложении во многом основаны на принципах работы системы YouGile. Основной акцент сделан на использовании досок задач, распределении ролей между участниками проекта и отображении этапов выполнения задач в визуальном формате.

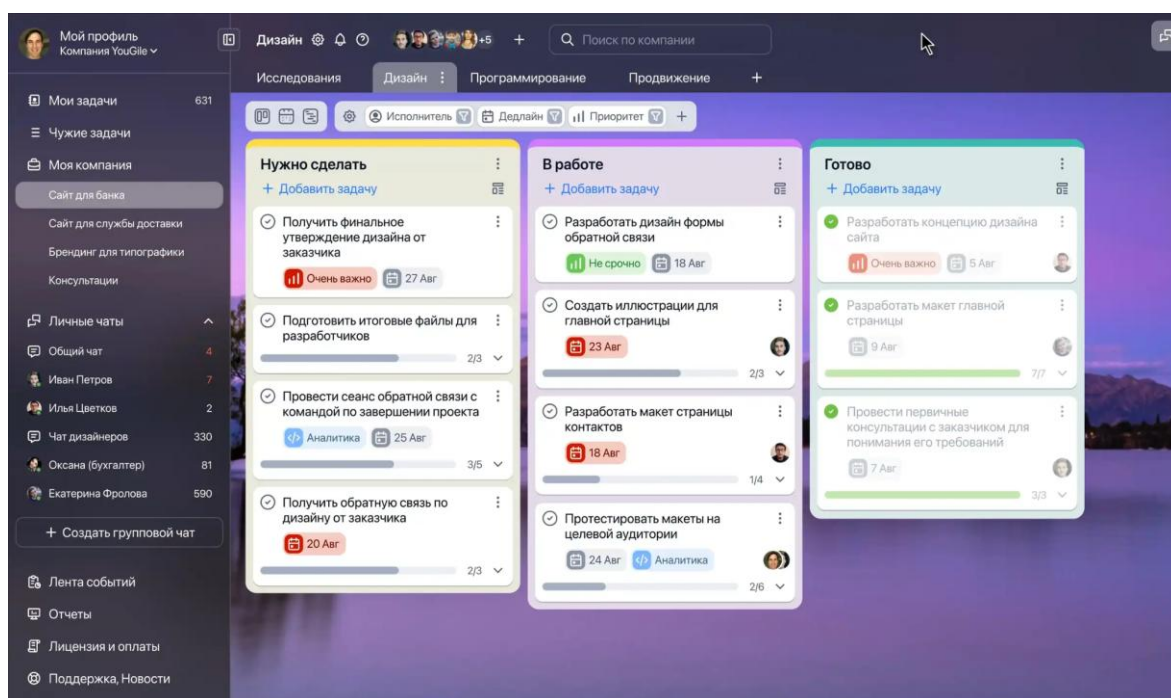


Рисунок 1.2 - Интерфейс системы YouGile

Другим распространенным решением является Trello. Данная система также использует kanban-доски для организации задач и предоставляет базовые средства для командного взаимодействия. Основными преимуществами системы являются простой интерфейс и удобная организация задач по этапам выполнения.

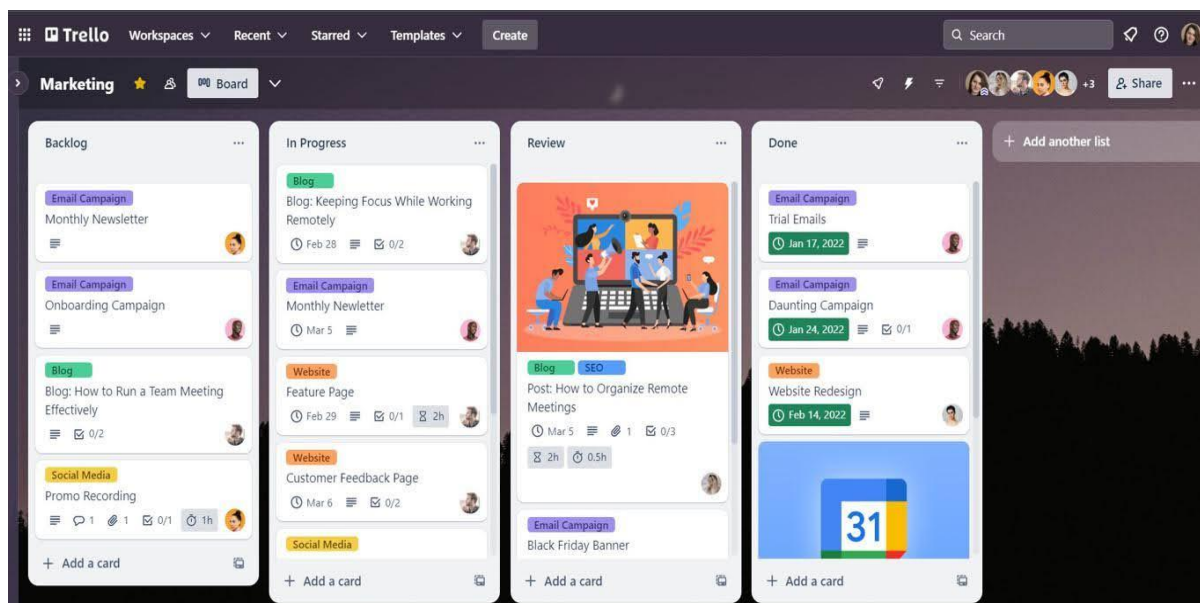


Рисунок 1.3 - Интерфейс системы Trello

Несмотря на наличие большого количества готовых решений, использование сторонних сервисов внутри компании имеет ряд ограничений. Большинство подобных систем являются универсальными платформами и не учитывают особенности внутренних процессов конкретной организации. Дополнительные проблемы возникают при необходимости изменения структуры приложения, внедрения нестандартной логики работы или интеграции с внутренними сервисами компании.

Использование сторонних решений также создает зависимость от внешних сервисов и ограничивает возможность полного контроля над данными компании. Часть функциональности может быть недоступна без приобретения платных подписок, а изменение архитектуры системы или

реализация специфических возможностей зачастую невозможны без вмешательства разработчиков самого сервиса.

Для тестового отдела разработки компании ООО “Интернет-Эксперт” требуется система, полностью адаптированная под внутренние процессы подразделения и особенности организации работы команды разработки. В связи с этим было принято решение о создании собственного desktop-приложения, реализующего необходимую функциональность и обеспечивающего возможность дальнейшего масштабирования системы.

Разработка собственного программного продукта позволяет:

- адаптировать функциональность системы под требования компании;
- реализовать необходимую структуру взаимодействия между сотрудниками;
- обеспечить полный контроль над хранимыми данными;
- внедрять новые возможности без ограничений сторонних сервисов;
- изменять архитектуру системы в процессе развития проекта;
- обеспечить интеграцию с внутренними сервисами компании.

Таким образом, разработка собственной системы управления проектами является более эффективным решением для организации работы тестового отдела разработки и позволяет учитывать особенности внутренних процессов компании без зависимости от сторонних платформ.

1.5 Обоснование выбора ПО для реализации проекта

Для реализации программного продукта был выбран стек технологий, обеспечивающий высокую производительность приложения, удобство разработки, поддержку многопользовательской работы и возможность дальнейшего масштабирования системы.

В качестве основного языка программирования серверной части был выбран язык Go. Данный язык активно используется при разработке

высоконагруженных сетевых приложений, серверных сервисов и распределенных систем. Основными преимуществами Go являются высокая скорость выполнения программного кода, встроенная поддержка конкурентного выполнения задач и относительно низкое потребление системных ресурсов.

Использование Go позволяет эффективно обрабатывать большое количество одновременных запросов, обеспечивать стабильную работу серверной части и поддерживать многопользовательское взаимодействие внутри системы. Дополнительным преимуществом языка является простота компиляции и развертывания приложений, что особенно важно при разработке корпоративных сервисов.

Еще одной причиной выбора Go является наличие встроенных средств работы с многопоточностью. Использование `goroutine` и каналов позволяет упростить реализацию параллельной обработки данных и организовать стабильную работу приложения при одновременном подключении нескольких пользователей.

Для организации взаимодействия между клиентской и серверной частью системы используется технология `gRPC`. Данная технология представляет собой современный механизм удаленного вызова процедур, разработанный компанией Google.

Основным преимуществом `gRPC` является высокая скорость обмена данными между компонентами системы. В отличие от традиционного REST-взаимодействия, `gRPC` использует бинарный формат передачи данных `Protocol Buffers`, позволяющий уменьшить объем передаваемой информации и повысить производительность приложения.

Использование `gRPC` также позволяет:

- упростить взаимодействие между сервисами;
- обеспечить строгую типизацию передаваемых данных;
- сократить количество ошибок при разработке API;

- повысить скорость обработки запросов;
- реализовать потоковую передачу данных между сервером и клиентом.

Для обновления информации в режиме реального времени в системе используются gRPC Stream-соединения. Использование потоковой передачи данных позволяет серверной части отправлять обновления клиенту без необходимости постоянной отправки повторных запросов. Благодаря этому пользователи могут оперативно получать изменения по задачам, комментариям и действиям других участников проекта.

В качестве основы для desktop-приложения был выбран фреймворк Wails. Данный фреймворк предназначен для разработки кроссплатформенных desktop-приложений с использованием языка Go и web-технологий.

Использование Wails позволяет объединить пользовательский интерфейс и серверную логику в единое приложение. Дополнительным преимуществом фреймворка является возможность создания desktop-приложений для различных операционных систем без необходимости значительного изменения структуры проекта.

Выбор desktop-приложения обусловлен необходимостью обеспечения стабильной работы системы и удобного взаимодействия пользователей с корпоративным программным обеспечением. Использование отдельного приложения позволяет снизить зависимость от браузера, повысить производительность интерфейса и обеспечить более тесную интеграцию с операционной системой пользователя.

Для хранения данных в разрабатываемой системе планируется использовать систему управления базами данных PostgreSQL. Данная СУБД является одной из наиболее распространенных систем хранения данных и широко применяется при разработке корпоративных информационных систем.

Выбор PostgreSQL обусловлен следующими преимуществами:

- высокая надежность хранения данных;
- поддержка сложных связей между сущностями;
- высокая производительность при работе с большими объемами информации;
- поддержка механизма транзакций;
- возможность масштабирования базы данных;
- наличие встроенных средств обеспечения целостности данных.

Предполагается, что использование PostgreSQL позволит организовать надежное хранение информации о пользователях, проектах, задачах, комментариях и других сущностях системы. Также данная СУБД обеспечит возможность выполнения сложных SQL-запросов и эффективной обработки данных.

Для проектирования пользовательского интерфейса системы планируется использовать сервис Figma. Данный инструмент предназначен для создания прототипов интерфейсов, проектирования структуры приложения и визуализации пользовательских сценариев.

Использование Figma позволит:

- разработать структуру пользовательского интерфейса;
- определить расположение элементов управления;
- создать макеты основных экранов системы;
- обеспечить единый стиль оформления интерфейса;
- упростить дальнейшую реализацию клиентской части приложения.

Предварительное проектирование интерфейса позволит определить структуру экранов системы и снизить вероятность внесения значительных изменений на этапе разработки.

Для реализации программного продукта планируется использовать среды разработки компании JetBrains:

- GoLand - для разработки серверной части и настольного приложения;
- DataGrip - для проектирования и сопровождения базы данных.

Среда разработки GoLand будет использоваться для написания программного кода на языке Go, отладки приложения, анализа структуры проекта и работы с системой контроля версий. Использование данной среды разработки позволит повысить эффективность разработки и упростить сопровождение программного продукта.

DataGrip планируется применять для проектирования структуры базы данных, создания таблиц, выполнения SQL-запросов и анализа данных. Использование специализированного инструмента позволит упростить взаимодействие с PostgreSQL и повысить удобство разработки базы данных.

Выбранный стек технологий должен обеспечить возможность дальнейшего развития системы, внедрения новых функциональных возможностей и поддержки многопользовательской работы без необходимости существенного изменения архитектуры программного продукта.

2 Проектная часть

2.1 Проектирование архитектуры проекта

Проектирование архитектуры информационной системы является ключевым этапом разработки программного продукта, на котором определяется структура приложения, взаимодействие компонентов, способы хранения и передачи данных, а также логика взаимодействия пользователей с системой.

Разрабатываемая система “Atlas” представляет собой многопользовательское desktop-приложение для организации командной работы, управления рабочими пространствами, kanban-досками и задачами. Система поддерживает потоковую синхронизацию данных с сервером, работу в фоновом режиме, разграничение прав доступа и совместную работу в реальном времени.

В качестве технологии desktop-клиента выбран фреймворк Wails [16], позволяющий создавать нативные приложения с использованием Go [1] и интерфейса на ReactJS. Он обеспечивает интеграцию веб-интерфейса с возможностями ОС, высокую производительность и низкое потребление ресурсов.

Клиентская часть реализована на ReactJS [17], что обеспечивает компонентный подход, удобное управление состоянием и динамическое обновление интерфейса без перезагрузки страницы.

Серверная часть разработана на Go. Для взаимодействия клиента и сервера используется gRPC [4], обеспечивающий высокую скорость обмена данными, эффективную сериализацию и поддержку потоковой передачи, что снижает нагрузку на сеть и повышает производительность системы.

Для хранения данных используется PostgreSQL [7], обеспечивающая надежность, поддержку транзакций, масштабируемость и эффективную работу со связанными данными.

При проектировании архитектуры системы были определены следующие основные требования:

- обеспечение многопользовательского режима работы;
- поддержка клиент-серверного взаимодействия;
- возможность потокового обновления данных;
- поддержка авторизации через электронную почту и сторонних провайдеров;
- разграничение прав доступа пользователей;
- обеспечение масштабируемости системы;
- обеспечение удобства сопровождения и расширения функциональности;
- обеспечение высокой производительности обмена данными;
- поддержка работы приложения в фоновом режиме.

Система реализована на основе клиент-серверной архитектуры, где клиент отвечает за интерфейс и взаимодействие с пользователем, а сервер - за бизнес-логику, обработку запросов и хранение данных.

Архитектура приложения включает следующие основные компоненты:

- desktop-клиент на Wails и ReactJS;
- gRPC-сервер приложений;
- база данных PostgreSQL;
- система потоковой синхронизации данных;
- модуль авторизации;
- модуль управления рабочими пространствами;
- модуль управления kanban-досками;
- модуль управления пользователями и ролями.

Клиент обеспечивает отображение интерфейса, отправку gRPC-запросов и получение обновлений в реальном времени, включая работу в фоновом режиме. Сервер выполняет обработку запросов, реализацию бизнес-

логики, проверку прав доступа и синхронизацию данных между участниками рабочих пространств.

Данные хранятся в PostgreSQL и включают информацию о пользователях, рабочих пространствах, ролях, приглашениях, досках, колонках и задачах.

Для синхронизации данных используются потоковые gRPC-соединения [5], благодаря которым изменения мгновенно отображаются у всех участников системы в реальном времени.

В системе реализована ролевая модель доступа: пользователь может состоять в нескольких рабочих пространствах и иметь разные роли:

- owner (Владелец);
- admin (Администратор);
- member (Участник).

Owner имеет полный доступ, включая удаление пространства и передачу прав. Admin управляет содержимым, member ограничен использованием функционала.

Сервер построен по многослойной архитектуре:

- gRPC-запросов;
- бизнес-логики;
- работы с данными;
- взаимодействия с базой данных.

Разделение на слои упрощает сопровождение и расширение системы без изменения архитектуры.

В процессе проектирования были разработаны диаграммы:

- DFD-диаграмма;
- IDEF0-диаграмма;
- Диаграмма классов;

- Диаграмма вариантов использования;
- Диаграмма последовательности;
- Диаграмма деятельности.

DFD-диаграмма

Отражает процесс обмена данными между пользователем и системой. Пользователь отправляет запросы через четыре интерфейсных модуля. Эти данные обрабатываются через шлюзы Protobuf и gRPC Stream, распределяются по внутренним сервисам системы и в итоге сохраняются в единой базе данных PostgreSQL.

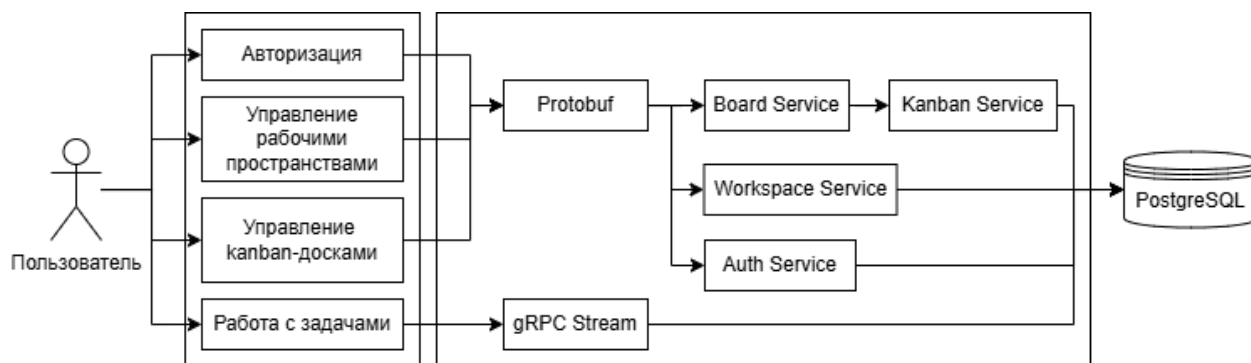


Рисунок 2.1.1 - DFD-диаграмма

IDEF0-диаграмма верхнего уровня

Отражает общее назначение информационной системы “Atlas”. На ней представлены основные входные данные, механизмы работы системы, управляющие воздействия и результаты обработки информации.



Рисунок 2.1.2 - IDEF0-диаграмма

IDEF0-диаграмма уровня A0

Диаграмма A0 показывает декомпозицию системы на основные функциональные подсистемы. Она позволяет определить взаимосвязи между ключевыми процессами, реализованными в информационной системе.

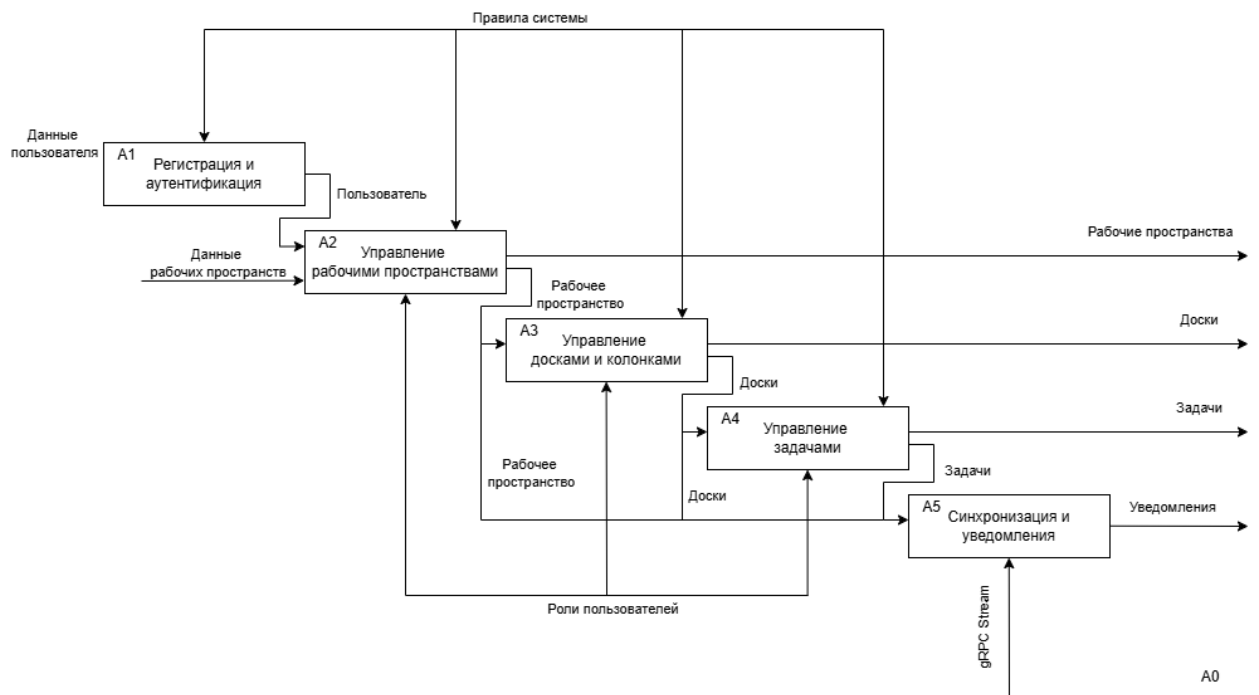


Рисунок 2.1.3 - IDEF0-диаграмма уровня A0

IDEF0-диаграмма уровня A1

Диаграмма A1 описывает процесс регистрации и аутентификации пользователей. В результате выполнения данного процесса пользователь получает доступ к функциональным возможностям системы.

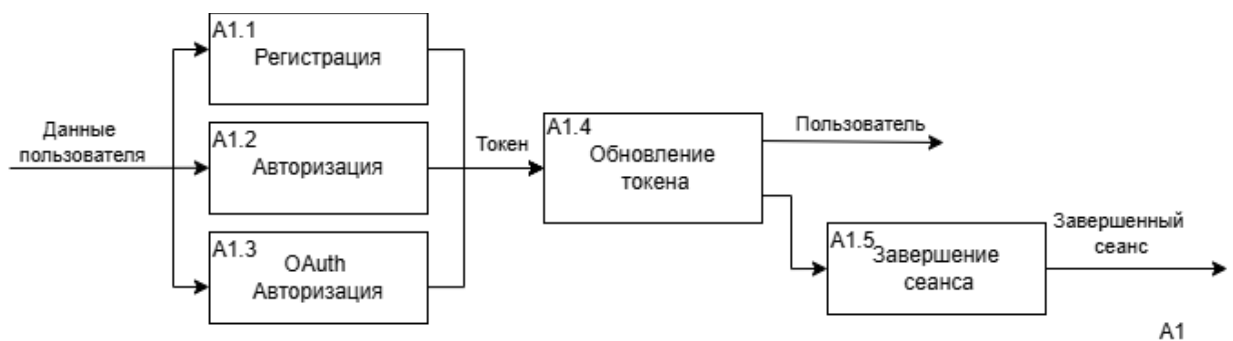


Рисунок 2.1.4 - IDEF0-диаграмма уровня A1

IDEF0-диаграмма уровня A2

Диаграмма A2 отображает процесс управления рабочим пространством. Она включает операции по созданию, изменению и контролю рабочего пространства.

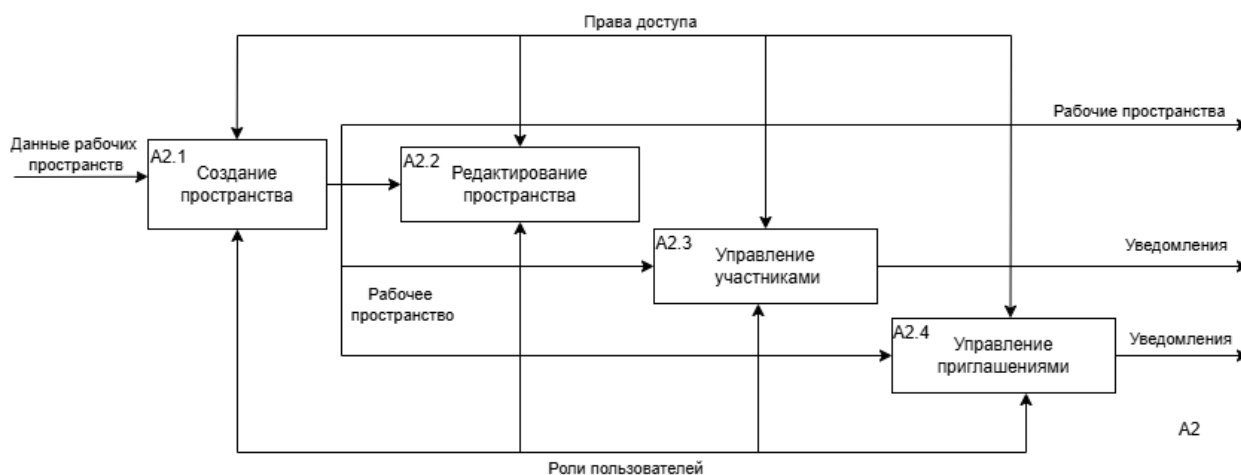


Рисунок 2.1.5 - IDEF0-диаграмма уровня A2

IDEF0-диаграмма уровня A3

Диаграмма A3 отображает процесс управления досками и колонками. Она включает операции по созданию, изменению и контролю досок и колонок.

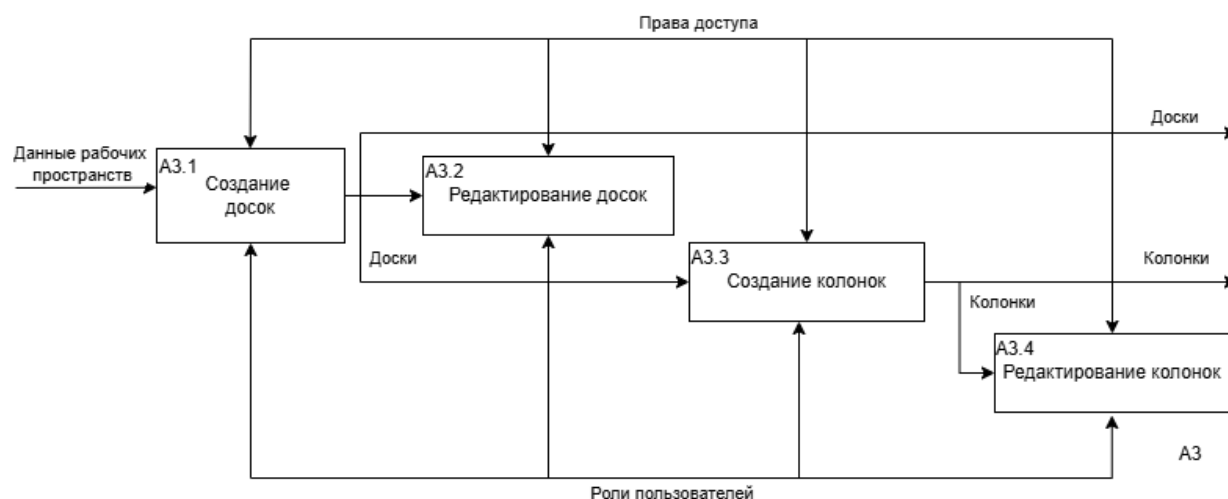


Рисунок 2.1.6 - IDEF0-диаграмма уровня A3

IDEF0-диаграмма уровня A4

Диаграмма A4 отображает процесс управления задачами и вложениями к ним. Она включает операции по созданию, изменению и контролю задач и их вложений.

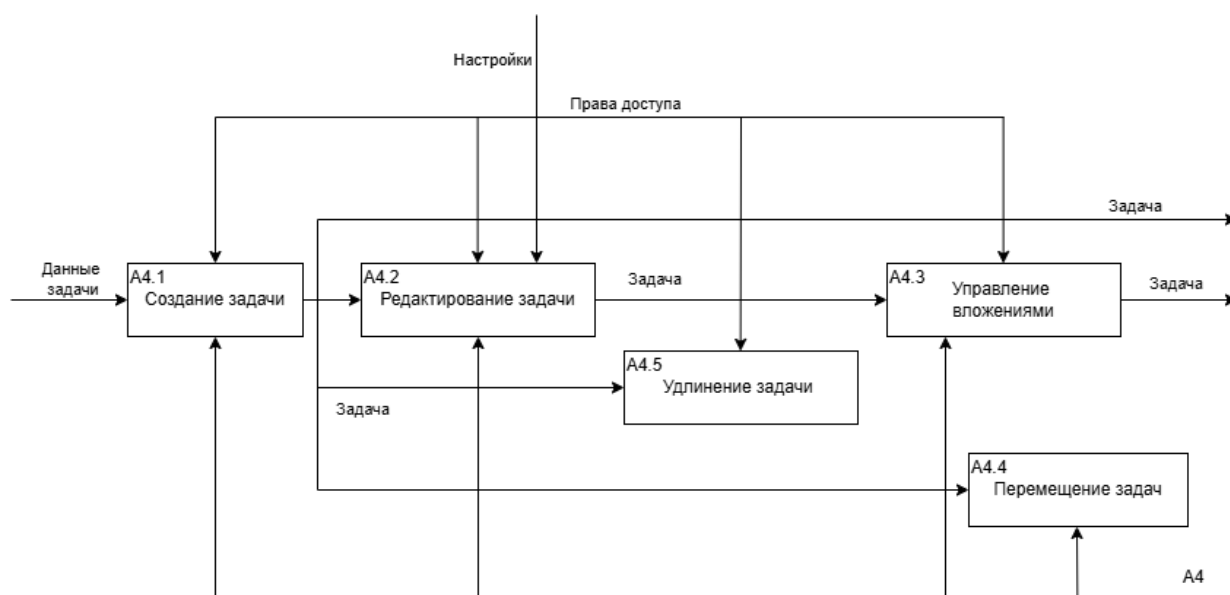


Рисунок 2.1.7 - IDEF0-диаграмма уровня A4

IDEF0-диаграмма уровня A5

Диаграмма A5 описывает процесс синхронизации информации. Реализация данного механизма обеспечивает доступ пользователей к необходимым данным.



Рисунок 2.1.8 - IDEF0-диаграмма уровня A5

Диаграмма классов

Диаграмма классов отображает структуру предметной области системы “Atlas”, включая основные сущности: User, Workspace, Boards, Column, Tasks, Workspace Members и Workspace Invite. На диаграмме представлены их атрибуты, методы и связи между объектами, что позволяет описать модель данных и взаимодействие сущностей внутри системы.

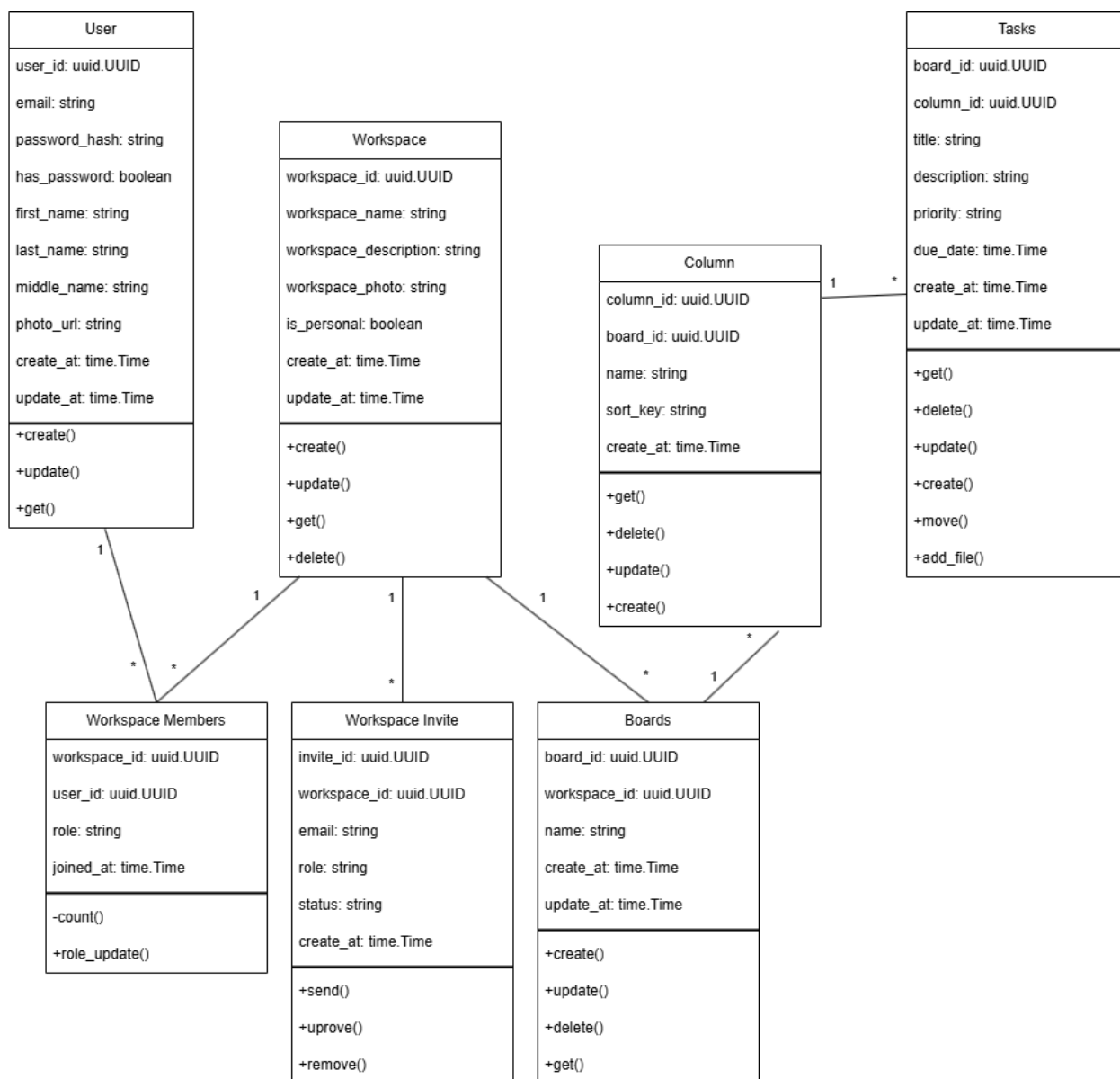


Рисунок 2.1.9 - Диаграмма классов

Диаграмма вариантов использования

Диаграмма вариантов использования отражает функциональные возможности системы “Atlas” с точки зрения пользователя. Она демонстрирует основные сценарии взаимодействия ролей Owner, Admin и Member с системой, включая авторизацию, создание и управление рабочими пространствами, досками и задачами, а также управление ролями, приглашениями и профилем пользователя.

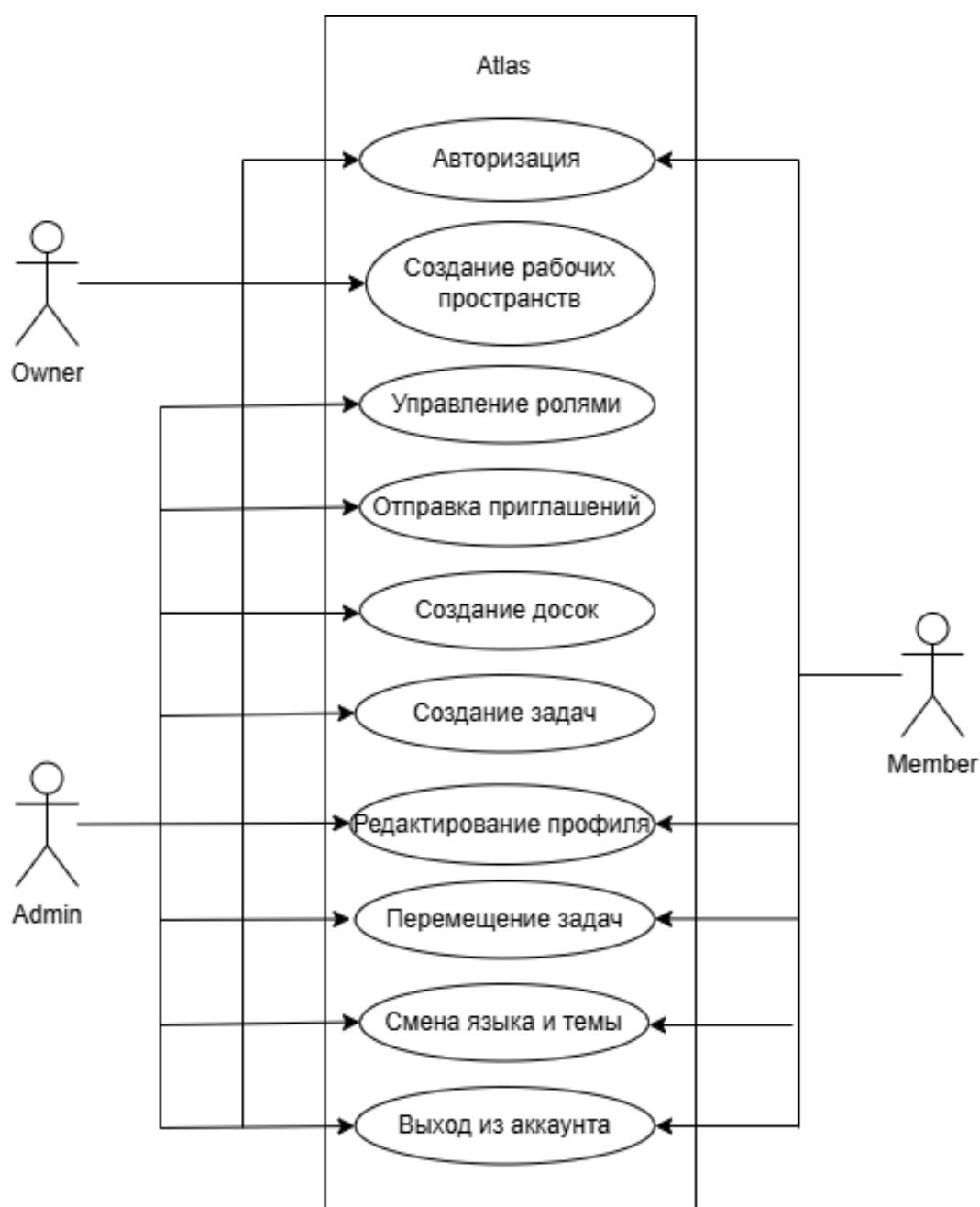


Рисунок 2.1.10 - Диаграмма вариантов использования

Диаграмма последовательности действий

Диаграмма последовательности демонстрирует порядок обмена сообщениями между компонентами системы при выполнении определенного сценария. Она позволяет проследить логику выполнения операций от начала до получения результата.

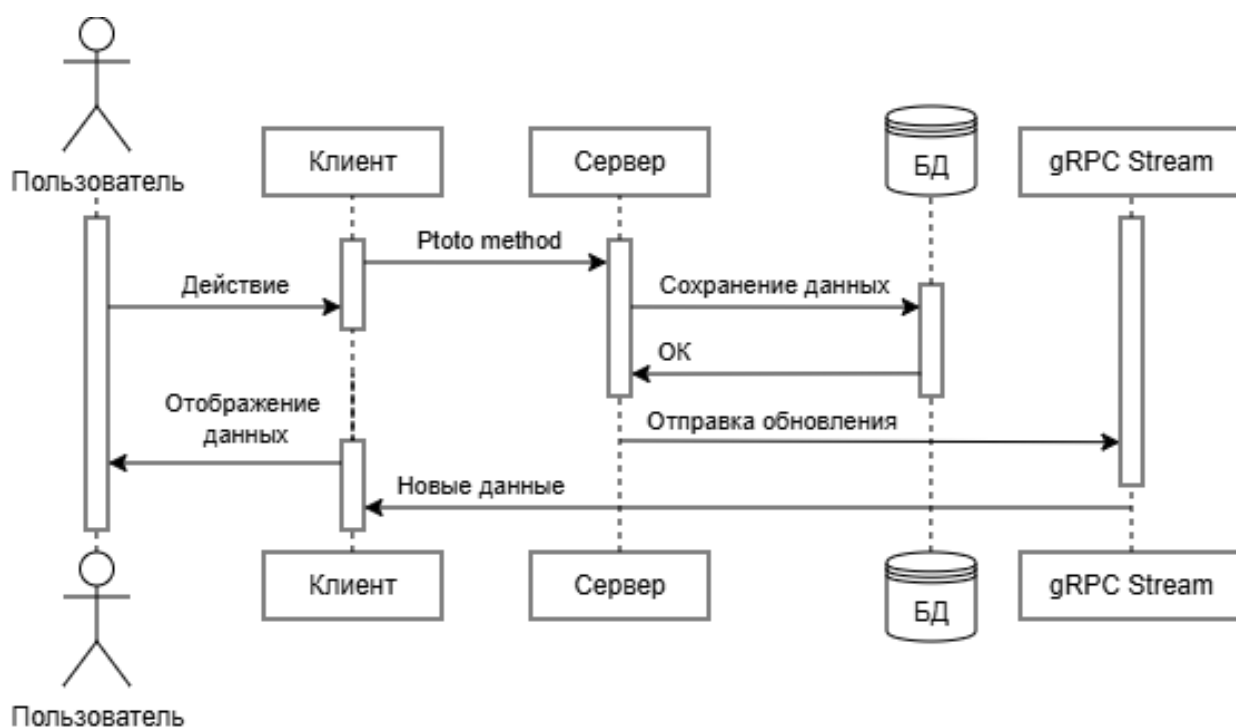


Рисунок 2.1.11 - Диаграмма последовательности действий

Диаграмма деятельности

Диаграмма деятельности отображает последовательность действий пользователя в системе Atlas от открытия приложения прохождения аутентификации и до начала использования функционала программы.

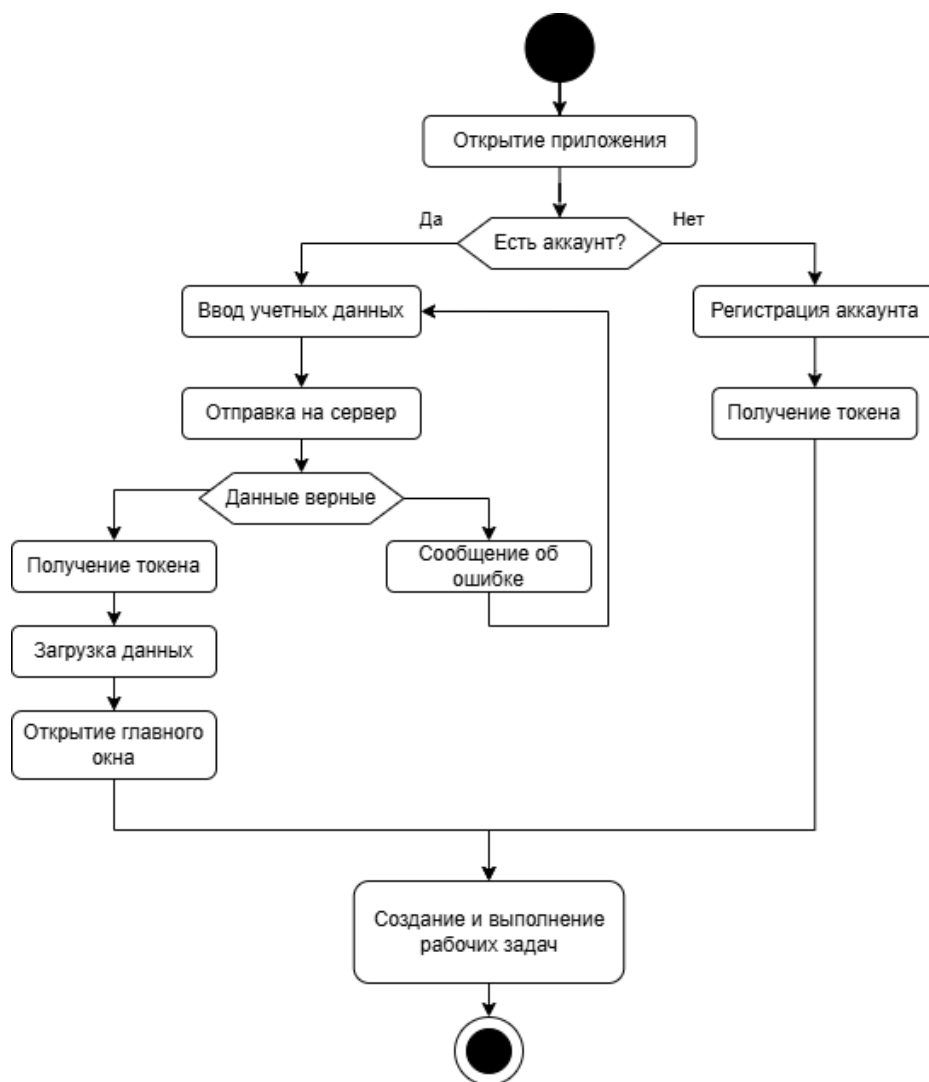


Рисунок 2.1.12 - Диаграмма деятельности

2.1.1 Проектирование базы данных

Для хранения информации в системе используется реляционная система управления базами данных PostgreSQL. Выбор PostgreSQL обусловлен высокой надежностью, поддержкой транзакций, механизмов индексации, расширяемостью и высокой производительностью при работе с многопользовательскими системами.

Структура базы данных была спроектирована с учетом требований многопользовательской работы, разграничения прав доступа, обеспечения

целостности данных и поддержки потоковой синхронизации изменений между клиентами системы.

В базе данных используются следующие основные таблицы:

- users;
- user_identities;
- workspaces;
- workspace_invites;
- workspace_members;
- boards;
- board_columns;
- tasks;
- task_attachments.

Взаимосвязи между основными сущностями базы данных и структура хранения информации представлены на рисунке 2.1.1.1. Диаграмма отражает состав таблиц базы данных, их ключевые атрибуты и связи, обеспечивающие целостность данных и корректное взаимодействие между отдельными модулями системы.

Разработанная структура базы данных позволяет организовать хранение информации о пользователях, рабочих пространствах, досках, задачах и связанных с ними объектах, а также обеспечивает поддержку многопользовательской работы и разграничения прав доступа.

Таблица users предназначена для хранения основной информации о пользователях системы. Каждый пользователь содержит уникальный адрес электронной почты, ФИО, ссылка на фотографию, дата создания и дата последнего обновления.

Таблица 2.1.1.1 - Таблица users в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	user_id	Уникальный идентификатор пользователя	UUID	PK
2	email	Электронная почта пользователя	TEXT	UNIQUE
3	password_hash	Захешированный пароль пользователя	TEXT	
4	first_name	Имя пользователя	TEXT	NOT NULL
5	last_name	Фамилия пользователя	TEXT	NOT NULL
6	middle_name	Отчество пользователя	TEXT	DEFAULT ''
7	photo_url	Ссылка на изображение	TEXT	DEFAULT ''
8	created_at	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()
9	updated_at	Дата и время последнего обновления	TIMESTAMPTZ	DEFAULT NOW ()

Таблица user_identities используется для хранения внешних способов авторизации пользователей через сторонних провайдеров. Данная таблица позволяет одному пользователю иметь несколько способов аутентификации.

Таблица 2.1.1.2 - Таблица user_identities в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	user_identity_id	Уникальный идентификатор способа авторизации	UUID	PK
2	user_id	Идентификатор пользователя	UUID	FK
3	provider	Наименование провайдера авторизации	TEXT	NOT NULL
4	provider_id	Уникальный идентификатор пользователя у провайдера	TEXT	NOT NULL
5	unique_provider_id	Уникальность комбинации provider и provider_id	CONSTRAINT	UNIQUE
6	unique_user_provider	Уникальность провайдера для одного пользователя	CONSTRAINT	UNIQUE

Для таблицы user_identities были реализованы ограничения уникальности unique_provider_id и unique_user_provider, обеспечивающие

невозможность повторного добавления одинакового провайдера авторизации и идентификатора пользователя.

Таблица `workspaces` предназначена для хранения рабочих пространств пользователей. Каждое рабочее пространство содержит название, описание, ссылка на изображение, тип рабочего пространства, дата создания и дата последнего обновления.

Таблица 2.1.1.3 - Таблица `workspaces` в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	<code>workspace_id</code>	Уникальный идентификатор рабочего пространства	UUID	PK
2	<code>name</code>	Название рабочего пространства	TEXT	NOT NULL
3	<code>description</code>	Описание рабочего пространства	TEXT	
4	<code>photo_url</code>	Ссылка на изображение	TEXT	NOT NULL
5	<code>is_personal</code>	Флаг, отвечающий за тип рабочего пространства	BOOLEAN	DEFAULT FALSE
6	<code>created_at</code>	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()
7	<code>updated_at</code>	Дата и время последнего обновления	TIMESTAMPTZ	DEFAULT NOW ()

Таблица `workspace_invites` содержит информацию о приглашениях пользователей в рабочие пространства. В таблице хранятся `uuid` рабочего пространства куда был приглашен пользователь, адрес электронной почты приглашенного пользователя, назначаемая роль и текущий статус приглашения.

Таблица 2.1.1.4 - Таблица `workspace_invites` в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	<code>invite_id</code>	Уникальный идентификатор приглашения	UUID	PK
2	<code>workspace_id</code>	Идентификатор рабочего пространства	UUID	FK
3	<code>email</code>	Электронная почта приглашенного пользователя	TEXT	NOT NULL

Продолжение таблицы 2.1.1.4

№	Псевдоним	Описание	Тип	Ключ
4	role	Роль приглашенного	WORKSPACE_ROLE	DEFAULT 'member'
5	status	Статус приглашения	WORKSPACE_INVITE_STATUS	DEFAULT 'pending'
6	created_at	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()

Таблица workspace_members реализует связь между пользователями и рабочими пространствами. В данной таблице хранится роль пользователя внутри конкретного рабочего пространства и дата присоединения к рабочему пространству.

Таблица 2.1.1.5 - Таблица workspace_members в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	workspace_id	Идентификатор рабочего пространства	UUID	PK, FK
2	user_id	Идентификатор пользователя	UUID	PK, FK
3	role	Роль пользователя	WORKSPACE_ROLE	DEFAULT 'member'
4	joined_at	Дата и время присоединения	TIMESTAMPTZ	DEFAULT NOW ()

Для таблицы workspace_members был реализован составной первичный ключ, включающий поля workspace_id и user_id. Это обеспечивает уникальность пользователя внутри конкретного рабочего пространства.

Таблица boards предназначена для хранения kanban-досок, принадлежащих определенным рабочим пространствам.

Таблица 2.1.1.6 - Таблица boards в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	board_id	Уникальный идентификатор доски	UUID	PK
2	workspace_id	Идентификатор рабочего пространства	UUID	FK
3	name	Имя доски	TEXT	NOT NULL
4	created_at	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()

Продолжение таблицы 2.1.1.6

№	Псевдоним	Описание	Тип	Ключ
5	updated_at	Дата и время последнего обновления	TIMESTAMPTZ	DEFAULT NOW ()

Таблица board_columns содержит колонки kanban-досок и информацию об их порядке отображения.

Таблица 2.1.1.7 - Таблица board_columns в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	column_id	Уникальный идентификатор колонки	UUID	PK
2	board_id	Идентификатор доски	UUID	FK
3	name	Имя колонки	TEXT	NOT NULL
4	sort_key	Ключ сортировки элементов внутри колонки	TEXT	DEFAULT NOW ()
5	created_at	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()

Таблица tasks используется для хранения задач. Для каждой задачи сохраняются название, описание, приоритет, крайний срок выполнения и принадлежность к определенной колонке.

Таблица 2.1.1.8 - Таблица tasks в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	task_id	Уникальный идентификатор задачи	UUID	PK
2	column_id	Идентификатор колонки	UUID	FK
3	title	Название задачи	TEXT	NOT NULL
4	description	Описание задачи	TEXT	DEFAULT 'member'
5	priority	Приоритет задачи	TASK_PRIORITY	DEFAULT 'medium'
6	due_date	Крайний срок выполнения	TIMESTAMPTZ	DEFAULT NULL
7	created_at	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()
8	updated_at	Дата и время последнего обновления	TIMESTAMPTZ	DEFAULT NOW ()

Таблица task_attachments предназначена для хранения файлов, прикрепленных к задачам.

Таблица 2.1.1.9 - Таблица task_attachments в базе данных

№	Псевдоним	Описание	Тип	Ключ
1	attachment_id	Уникальный идентификатор вложения	UUID	PK
2	task_id	Идентификатор задачи	UUID	FK
3	user_id	Идентификатор пользователя	UUID	FK
4	file_name	Название файла	TEXT	
5	file_url	Ссылка на файл	TEXT	
6	file_type	Тип файла	TEXT	
7	file_size	Размер файла	BIGINT	
8	created_at	Дата и время создания	TIMESTAMPTZ	DEFAULT NOW ()

Для повышения производительности системы во всех таблицах базы данных были созданы индексы для быстрого поиска и фильтрации данных.

Индексация используется для:

- идентификаторов пользователей;
- адресов электронной почты;
- идентификаторов рабочих пространств;
- идентификаторов досок;
- идентификаторов колонок;
- идентификаторов задач;
- ролей пользователей;
- статусов приглашений;
- внешних идентификаторов авторизации.

В базе данных также используются пользовательские типы данных PostgreSQL:

- workspace_role;
- workspace_invite_status;
- task_priority.

Тип `workspace_role` используется для хранения ролей пользователей внутри рабочих пространств. Поддерживаются следующие значения:

- `owner`;
- `admin`;
- `member`.

Тип `workspace_invite_status` используется для хранения состояния приглашения пользователя в рабочее пространство. Возможные значения:

- `pending`;
- `accepted`;
- `rejected`;
- `expired`.

Тип `task_priority` используется для хранения уровня приоритета задачи. Возможные значения:

- `low`;
- `medium`;
- `high`;
- `critical`.

Для автоматического обновления поля `updated_at` во всех таблицах базы данных используется специальная PostgreSQL-функция и триггеры. При изменении записи поле `updated_at` автоматически обновляется текущими датой и временем.

Используемая функция:



```
create or replace function set_updated_at()  
returns trigger as $$  
begin  
    new.updated_at = now();  
    return new;  
end;  
$$ language plpgsql;
```

Рисунок 2.1.1.1 - SQL функция обновления

Для автоматического создания триггеров для всех таблиц, содержащих поле `updated_at`, используется следующий SQL-скрипт:



```
do $$  
declare r record;  
begin  
    for r in  
        select table_name  
        from information_schema.columns  
        where column_name = 'updated_at'  
        and table_schema = 'public'  
    loop  
        execute format(  
            'drop trigger if exists trg_updated_at on %I;  
            create trigger trg_updated_at  
            before update on %I  
            for each row execute function set_updated_at();',  
            r.table_name,  
            r.table_name  
        );  
    end loop;  
end $$;
```

Рисунок 2.1.1.2 - SQL функция создания триггеров

Использование триггеров позволяет централизованно управлять обновлением времени изменения записей и исключает необходимость ручного обновления поля `updated_at` в серверной логике приложения.

Связи между таблицами реализованы при помощи внешних ключей, обеспечивающих целостность данных и корректность связей между сущностями системы. Структура базы данных была нормализована для минимизации дублирования данных и упрощения сопровождения системы.

2.2 Разработка интерфейсов

Разработка пользовательского интерфейса являлась одним из ключевых этапов создания информационной системы “Atlas”, поскольку именно через интерфейс осуществляется взаимодействие пользователей с проектами, рабочими пространствами, задачами и другими элементами системы. Основной целью разработки являлось создание удобной рабочей среды, позволяющей сотрудникам быстро получать доступ к необходимой информации и выполнять основные операции с минимальным количеством действий.

При проектировании интерфейса особое внимание уделялось удобству навигации, визуальной понятности элементов управления и скорости выполнения типовых операций. Поскольку система предназначена для ежедневного использования сотрудниками организации, важным требованием являлось снижение времени на выполнение рутинных действий, связанных с постановкой задач, контролем их выполнения и взаимодействием между участниками проекта.

После запуска приложения пользователю отображается окно авторизации. Для получения доступа к системе необходимо выполнить вход с использованием адреса электронной почты и пароля либо воспользоваться авторизацией через учетную запись Google. Если пользователь еще не

зарегистрирован в системе, ему доступна форма создания новой учетной записи.

После успешной авторизации пользователь автоматически перенаправляется в основное рабочее окно приложения. Центральное место в интерфейсе занимает область управления проектами и задачами, а основные элементы навигации расположены в боковой панели.

В левой части интерфейса располагается навигационная панель, обеспечивающая доступ ко всем основным разделам системы. Через данную панель пользователь может перейти к списку рабочих пространств, открыть kanban-доски, изменить параметры приложения или просмотреть информацию о собственном профиле.

Основными разделами системы являются:

- рабочие пространства;
- kanban-доски;
- настройки;
- профиль пользователя.

Использование постоянной боковой панели позволяет пользователю быстро переключаться между разделами без необходимости возврата на главную страницу приложения. При необходимости панель может быть свернута, что позволяет увеличить доступную рабочую область и обеспечить более комфортную работу с содержимым kanban-досок.

После входа в систему пользователь попадает в раздел рабочих пространств. Рабочее пространство представляет собой отдельную область для организации командной работы над проектом. Внутри рабочего пространства располагаются доски, задачи, участники и связанные с ними данные.

Каждое рабочее пространство отображается в виде отдельной карточки, содержащей основную информацию о проекте. Пользователь может

просмотреть название пространства, его описание, тип пространства и собственную роль в данном проекте. Такой способ представления информации позволяет быстро определить необходимые проекты и перейти к работе с ними.

Для создания нового рабочего пространства используется специальная кнопка создания проекта. После её нажатия открывается окно ввода параметров рабочего пространства. Пользователь указывает название проекта, его описание и при необходимости загружает изображение, которое будет использоваться в качестве визуального идентификатора пространства.

После создания рабочего пространства пользователь автоматически получает права владельца проекта. Это позволяет ему приглашать новых участников, управлять ролями пользователей, создавать доски и изменять параметры рабочего пространства.

Для организации совместной работы реализован механизм приглашения участников. Пользователь с соответствующими правами может открыть раздел приглашений, указать адрес электронной почты сотрудника и выбрать роль, которая будет назначена после вступления в проект. Отправленные приглашения отображаются в отдельном списке и сохраняются до момента принятия или отклонения пользователем.

После принятия приглашения новый участник автоматически появляется в списке участников рабочего пространства. Владельцы и администраторы проекта могут просматривать список пользователей, изменять их роли и контролировать состав команды. Такой подход позволяет централизованно управлять доступом к проекту и обеспечивать разграничение прав пользователей.

Основным инструментом управления рабочим процессом в системе являются kanban-доски. Каждая доска предназначена для визуального отображения текущего состояния проекта и содержит набор колонок, отражающих различные этапы выполнения работ.

После открытия доски пользователю отображается рабочая область, состоящая из колонок и карточек задач. Колонки позволяют разделить задачи по этапам выполнения. Подобный подход обеспечивает наглядное представление текущего состояния проекта и позволяет быстро оценить объем выполненной и оставшейся работы.

Создание новых колонок выполняется непосредственно на странице доски. Пользователь вводит название колонки и подтверждает её создание. Аналогичным образом осуществляется создание новых задач. Для постановки задачи достаточно указать её название, после чего карточка автоматически появляется в выбранной колонке.

Каждая задача представлена в виде отдельной карточки, содержащей краткую информацию о выполняемой работе. В карточке отображаются название задачи, установленный приоритет и крайний срок выполнения. Благодаря этому пользователь может быстро оценить важность задачи и определить очередность её выполнения.

Для просмотра подробной информации используется карточка задачи, открываемая в отдельном окне. В данном окне отображается полное описание задачи, сведения об исполнителях, установленный срок выполнения, прикрепленные файлы и другая дополнительная информация.

Особое внимание при разработке интерфейса уделялось удобству изменения состояния задач. Для этого реализован механизм перетаскивания карточек между колонками. Пользователь может перемещать задачу мышью из одной колонки в другую, изменяя тем самым её текущее состояние. Во время перемещения система визуально выделяет колонку назначения, что облегчает выполнение операции и снижает вероятность ошибок.

Использование механизма Drag-and-Drop позволяет отказаться от дополнительных форм изменения статуса задачи и значительно ускоряет работу пользователей с системой. Изменение положения карточки автоматически сохраняется и становится доступным всем участникам проекта.

Для хранения дополнительных материалов реализован механизм прикрепления файлов к задачам. Пользователь может загрузить документы, изображения и другие файлы, необходимые для выполнения работы. Все вложения отображаются непосредственно в карточке задачи и доступны участникам рабочего пространства.

Важной частью интерфейса является раздел профиля пользователя. В данном разделе отображаются основные сведения об учетной записи, включая фотографию пользователя, фамилию, имя, отчество и адрес электронной почты. При необходимости пользователь может изменить свои данные с помощью встроенного механизма редактирования профиля.

Дополнительно в приложении реализован раздел настроек, предназначенный для персонализации интерфейса. Пользователь может выбрать язык интерфейса, переключаться между светлой и темной темой оформления, а также выполнять выход из учетной записи. Сохранение настроек осуществляется автоматически, благодаря чему выбранные параметры восстанавливаются при последующих запусках приложения.

Для повышения удобства работы реализована система уведомлений. Уведомления используются для информирования пользователя о результатах выполненных действий, успешном создании объектов, возникновении ошибок и других событиях системы. Благодаря этому пользователь получает своевременную обратную связь о выполняемых операциях.

Одной из важных особенностей интерфейса является поддержка обновления данных в режиме реального времени. При изменении задач, досок или рабочих пространств информация автоматически обновляется у всех подключенных участников проекта. Пользователям не требуется вручную обновлять страницу или повторно открывать разделы приложения для получения актуальных данных.

В результате разработки был создан современный пользовательский интерфейс, обеспечивающий удобную работу с проектами, задачами и участниками. Реализованные решения позволяют эффективно организовать

командную деятельность, повысить удобство взаимодействия пользователей с системой и сократить время выполнения основных операций.

2.3 Разработка клиентской части

Клиентская часть информационной системы “Atlas” реализована в виде desktop-приложения на основе фреймворка Wails [16], обеспечивающего взаимодействие между серверной логикой, реализованной на языке Go [1], и пользовательским интерфейсом, разработанным с использованием ReactJS [17]. Использование данного подхода позволило объединить возможности нативного настольного приложения с удобством современных веб-интерфейсов.

Основной задачей клиентской части является обеспечение взаимодействия пользователя с рабочими пространствами, участниками проектов, kanban-досками и задачами. Интерфейс приложения предоставляет пользователю полный набор инструментов для организации командной работы и контроля выполнения поставленных задач.

После запуска приложения пользователю отображается окно авторизации. Для входа в систему необходимо указать адрес электронной почты и пароль либо воспользоваться авторизацией через Google [12]. При отсутствии учетной записи пользователь может перейти к форме регистрации и создать новый аккаунт. На рисунке 2.3.1 представлена форма авторизации пользователя.

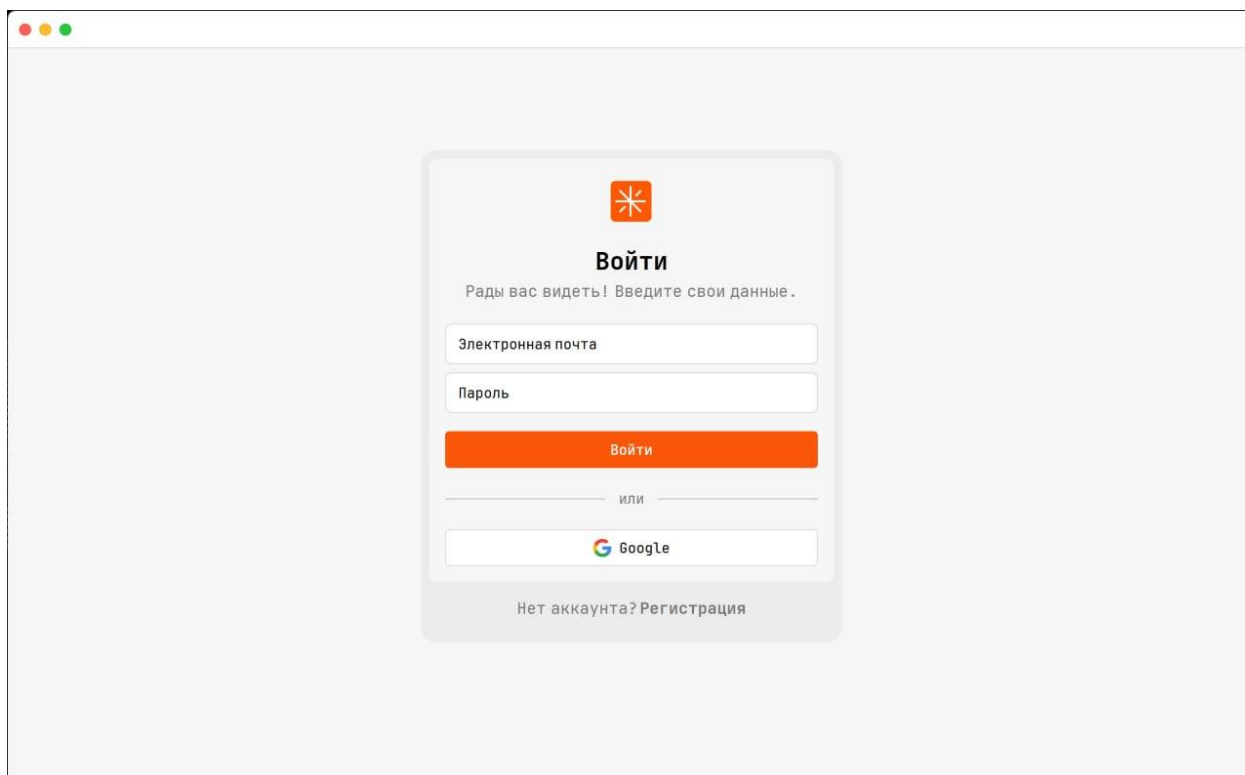


Рисунок 2.3.1 - Страница авторизации

После ввода учетных данных выполняется обращение к серверной части приложения для проверки корректности введенной информации. В случае успешной авторизации пользователю выдается JWT-токен [14], который используется для выполнения последующих запросов к серверу без необходимости повторного ввода учетных данных. На рисунке 2.3.2 представлен фрагмент программного кода, реализующий процесс авторизации пользователя.



```

func (s *Service) Login(email, password string) (map[string]interface{}, error) {
    resp, err := s.grpc.Auth.Login(
        context.Background(),
        &pb.LoginRequest{
            Email:    email,
            Password: password,
        },
    )

    if err != nil {
        return nil, err
    }

    s.state.Token = resp.Token
    s.state.CurrentUser = resp.User

    _ = s.saveUserLocally(resp.User, resp.Token)

    go s.stream.Start()

    return map[string]interface{}{
        "user":    resp.User,
        "success": true,
    }, nil
}

```

Рисунок 2.3.2 - Фрагмент кода процесса авторизации

После успешной авторизации выполняется загрузка пользовательских данных и открывается основное окно приложения. Если пользователь выполняет вход впервые, система автоматически создает персональное рабочее пространство, которое используется для индивидуальной работы и хранения личных задач. На рисунке 2.3.3 представлен интерфейс после успешной авторизации пользователя.

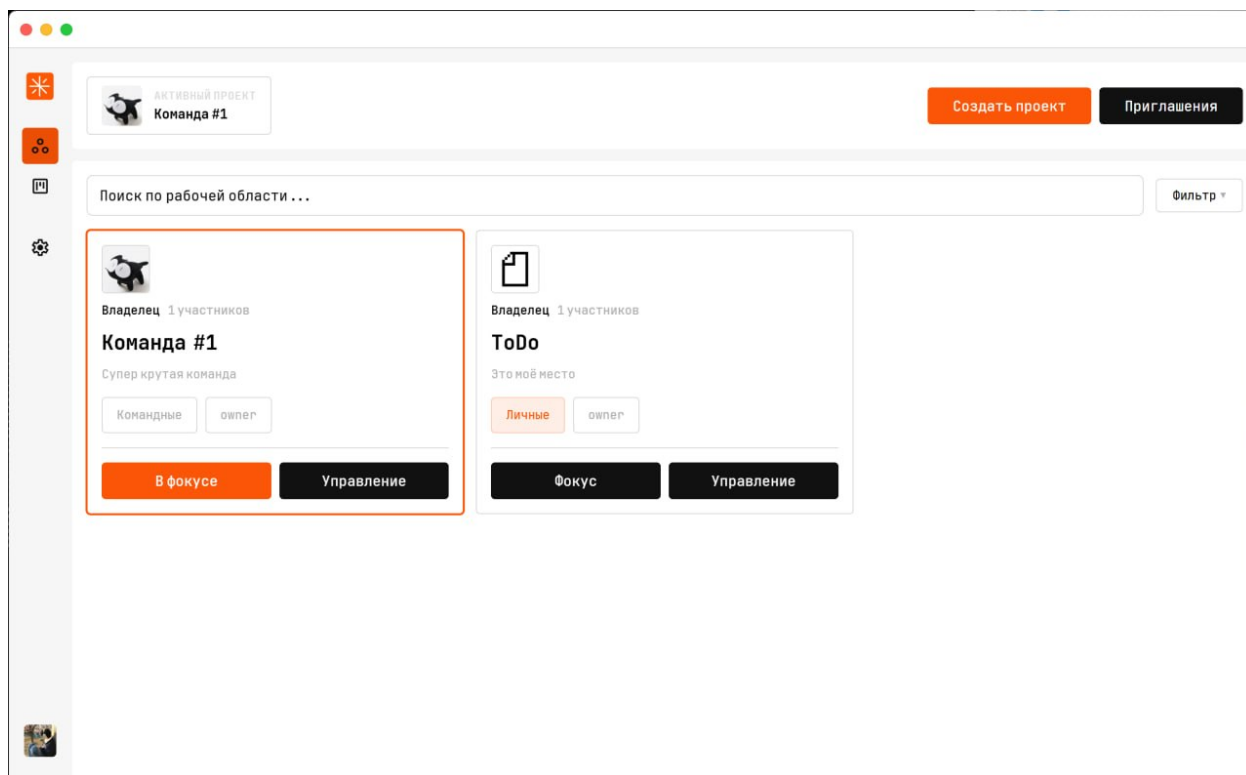


Рисунок 2.3.3 - Главная страница

Основным разделом приложения является раздел рабочих пространств. Рабочее пространство представляет собой отдельную область для организации работы над проектом. Внутри рабочего пространства хранятся доски, задачи, участники проекта и связанные с ними данные.

После открытия раздела пользователю отображается список доступных рабочих пространств. Для каждого пространства выводится его название, описание, тип проекта и роль текущего пользователя. Такой подход позволяет быстро определить необходимый проект и перейти к работе с ним.

Для создания нового рабочего пространства используется кнопка создания проекта. После её нажатия открывается специальное окно, содержащее форму ввода параметров нового пространства. На рисунке 2.3.4 представлено окно создания рабочего пространства.

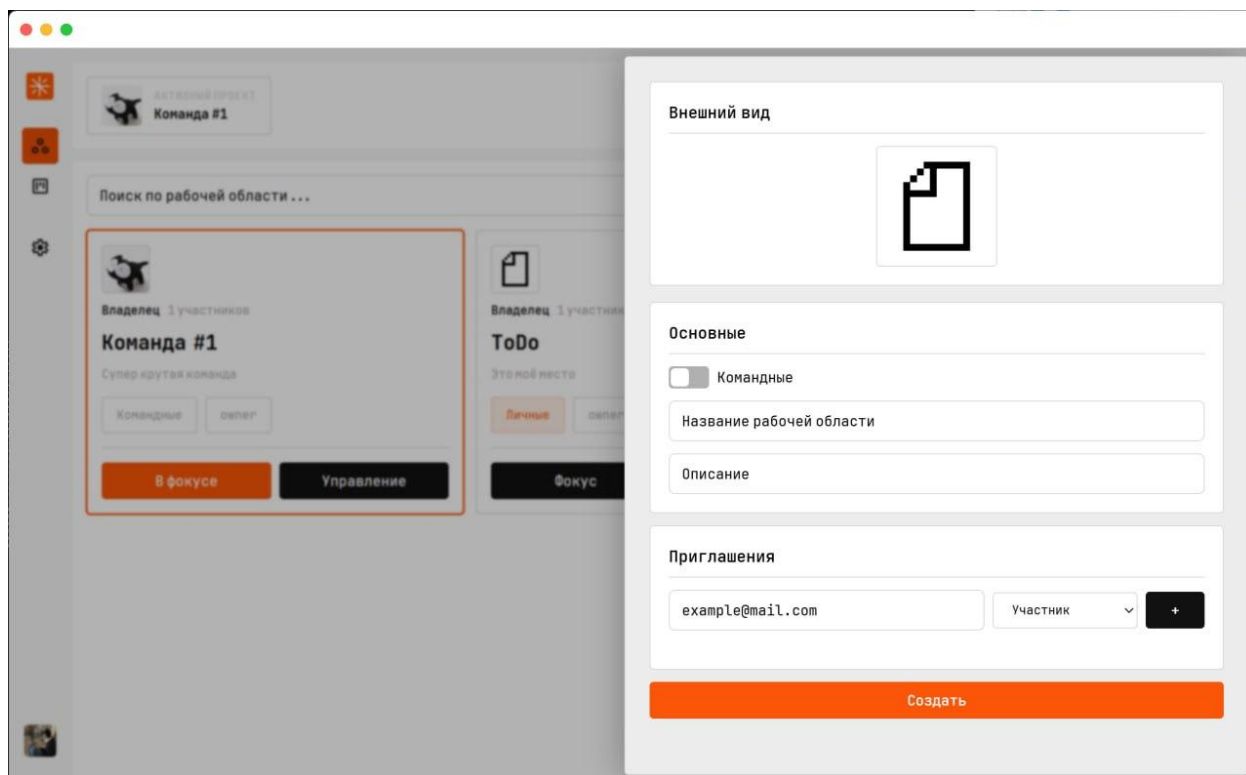


Рисунок 2.3.4 - Форма создания рабочего пространства

При создании рабочего пространства пользователь указывает название проекта, описание и при необходимости загружает изображение. После подтверждения операции выполняется отправка данных на сервер, где создается новая запись рабочего пространства. На рисунке 2.3.5 представлен фрагмент программного кода, отвечающий за создание рабочего пространства.

```

func (s *Service) CreateWorkspace(name string, description string, photoUrl string, isPersonal bool, inviteInputs
]InviteInput) (*pb2.Workspace, error) {
    if s.state.Token == "" {
        return nil, fmt.Errorf("user not authenticated")
    }

    req := &pb2.CreateWorkspaceRequest{
        Name:      name,
        Description: description,
        IsPersonal: isPersonal,
    }

    if photoUrl != "" {
        req.PhotoUrl = &photoUrl
    }

    for _, inv := range inviteInputs {
        req.Invites = append(
            req.Invites,
            &pb2.WorkspaceInviteInput{
                Email: inv.Email,
                Role:  pb2.WorkspaceRole(inv.Role),
            },
        )
    }

    resp, err := s.grpc.Workspace.CreateWorkspace(
        context.Background(),
        req,
    )

    if err != nil {
        return nil, fmt.Errorf(
            "failed to create workspace: %w",
            err,
        )
    }

    return resp, nil
}

```

Рисунок 2.3.5 - Фрагмент кода создания рабочего пространства

После успешного завершения операции проект автоматически появляется в списке доступных пространств, а пользователю назначается роль владельца.

Для управления проектом используется окно настроек рабочего пространства. В данном окне отображается основная информация о проекте, включая уникальный идентификатор пространства, дату создания, количество участников и сведения о владельце. На рисунке 2.3.6 представлен интерфейс управления рабочим пространством.

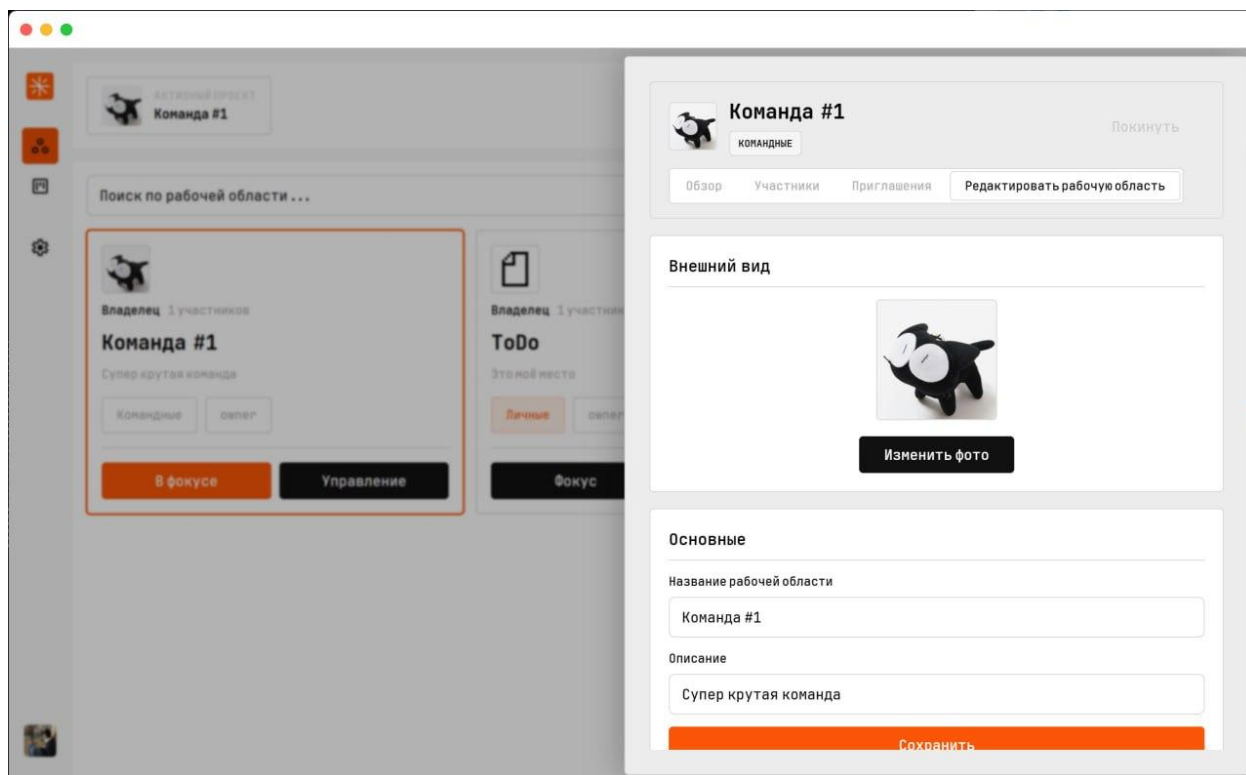


Рисунок 2.3.6 - Форма редактирования рабочего пространства

Для организации совместной работы реализован механизм приглашения участников. Пользователь с ролью владельца или администратора может открыть вкладку приглашений и отправить приглашение новому сотруднику. На рисунке 2.3.7 представлен интерфейс отправки приглашения.

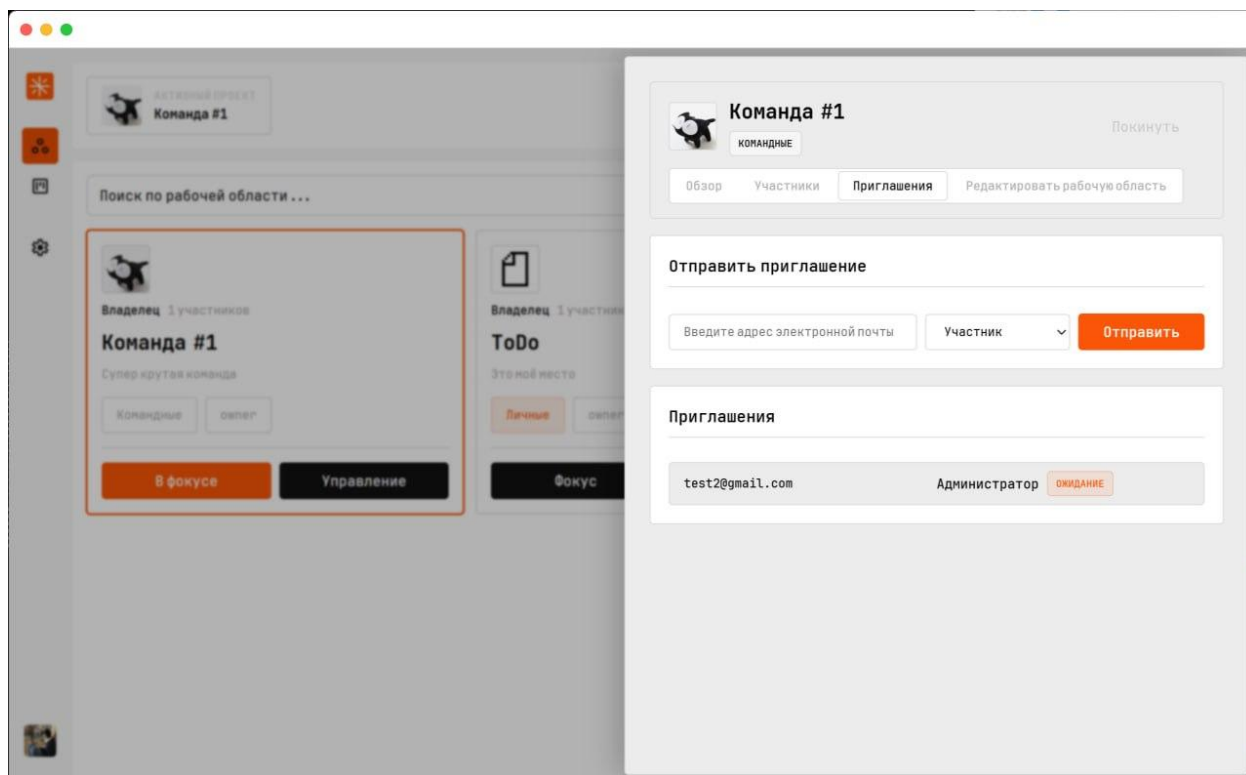


Рисунок 2.3.7 - Меню приглашений в рабочее пространство

При создании приглашения необходимо указать адрес электронной почты пользователя и выбрать роль, которая будет назначена после вступления в проект. На рисунке 2.3.8 представлен фрагмент программного кода, реализующий создание приглашения в рабочее пространство.



```

func (s *Service) CreateInvite(workspaceID string, email string, role int32) (*pb2.WorkspaceInvite, error) {
    if s.state.Token == "" {
        return nil, fmt.Errorf("user not authenticated")
    }

    req := &pb2.CreateInviteRequest{
        WorkspaceId: workspaceID,
        Email:       email,
        Role:        pb2.WorkspaceRole(role),
    }

    resp, err := s.grpc.WorkspaceInvite.CreateInvite(
        context.Background(),
        req,
    )

    if err != nil {
        if st, ok := status.FromError(err); ok {
            return nil, fmt.Errorf(st.Message())
        }

        return nil, err
    }

    return resp, nil
}

```

Рисунок 2.3.8 - Фрагмент кода создания приглашения в рабочее пространство

После отправки приглашения информация сохраняется в системе и отображается в списке активных приглашений. Получив приглашение, пользователь может ознакомиться с информацией о рабочем пространстве и принять либо отклонить приглашение. На рисунке 2.3.9 представлен интерфейс подтверждения приглашения.

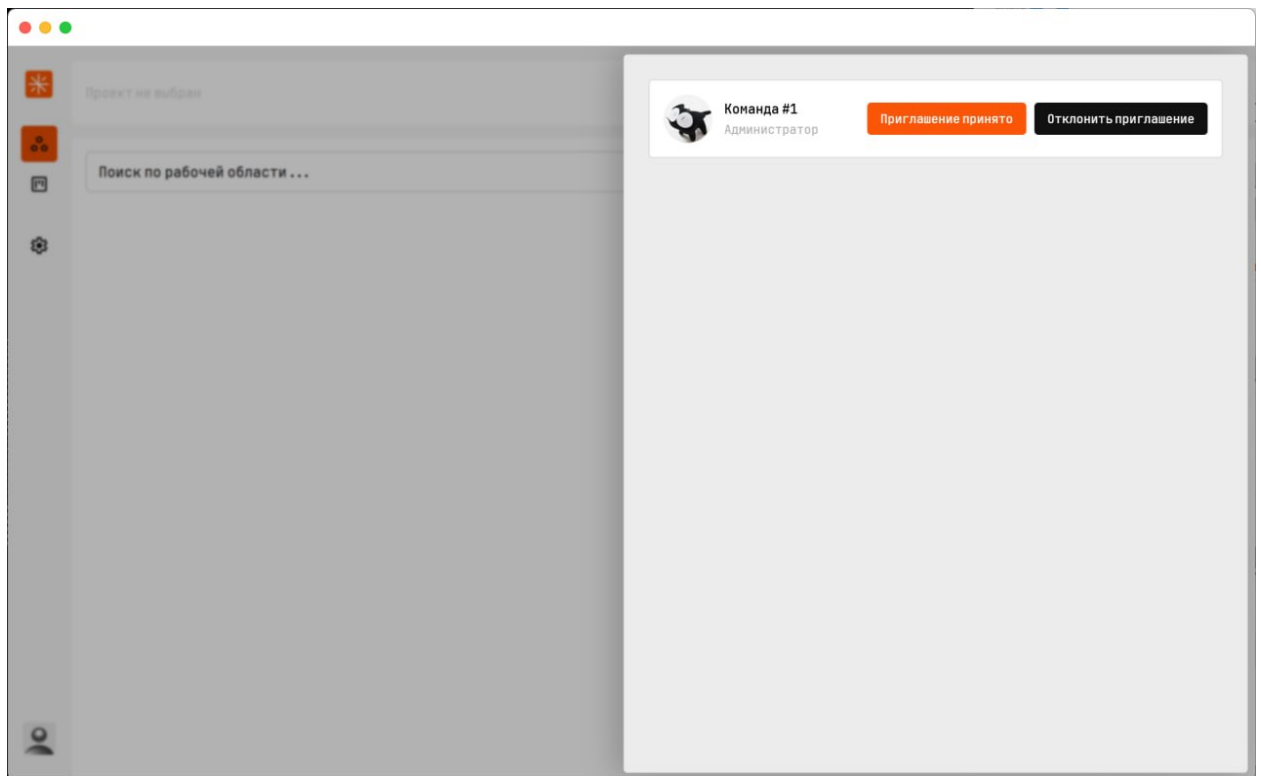


Рисунок 2.3.9 - Панель приглашений

На рисунке 2.3.10 представлен фрагмент программного кода, реализующий подтверждение приглашения пользователем.

```
func (s *Service) JoinWorkspace(workspaceID string) (*pb2.WorkspaceMember, error) {  
    if s.state.Token == "" {  
        return nil, fmt.Errorf("user is not auth")  
    }  
  
    resp, err := s.grpc.Workspace.JoinWorkspace(  
        context.Background(),  
        &pb2.JoinWorkspaceRequest{  
            WorkspaceId: workspaceID,  
        },  
    )  
  
    if err != nil {  
        return nil, err  
    }  
  
    return resp, nil  
}
```

Рисунок 2.3.10 - Фрагмент кода подтверждения приглашения

После подтверждения приглашения пользователь автоматически добавляется в рабочее пространство и появляется в списке участников проекта. На рисунке 2.3.11 представлен список участников рабочего пространства.

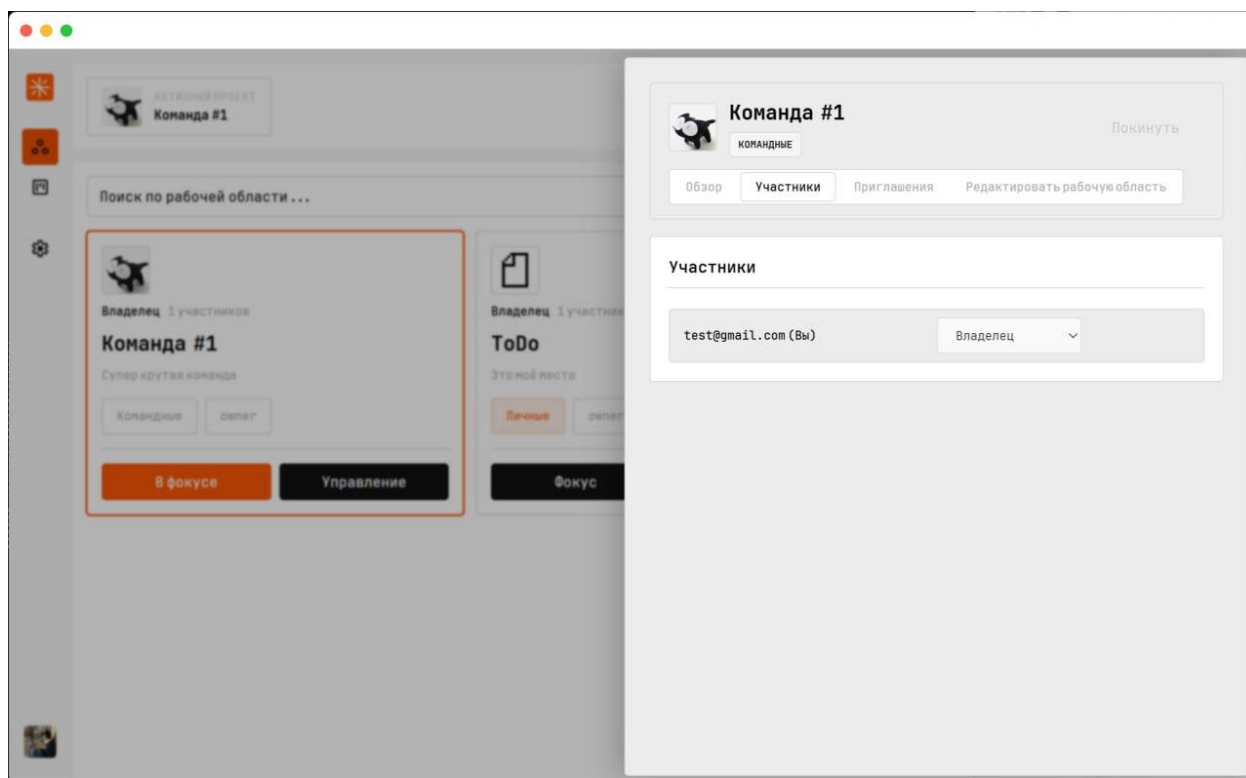


Рисунок 2.3.11 - Список участников в рабочем пространстве

Для управления рабочими задачами внутри рабочего пространства используются kanban-доски [25]. Каждая доска предназначена для визуального представления процесса выполнения работ и содержит набор колонок, соответствующих различным этапам жизненного цикла задачи. На рисунке 2.3.12 представлен список kanban-досок рабочего пространства.

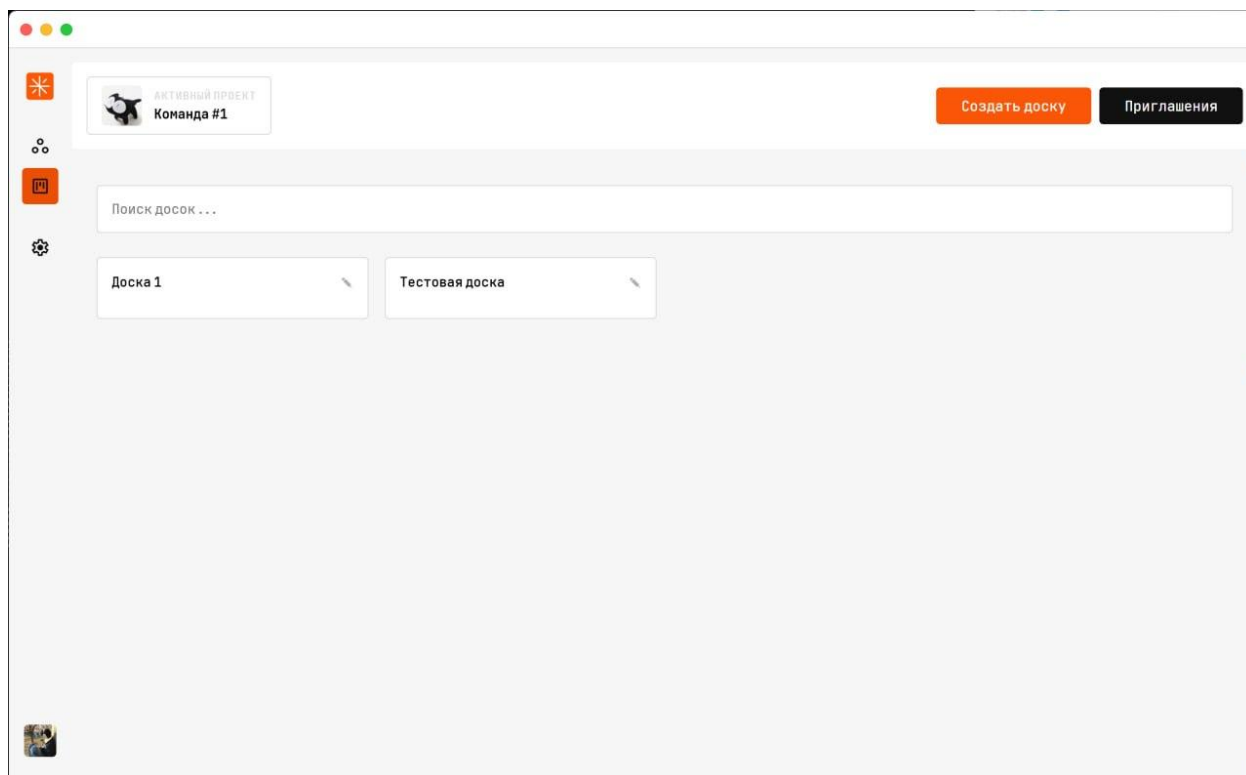


Рисунок 2.3.12 - Список kanban-досок

Создание новой доски осуществляется через специальную форму. Пользователь указывает название доски, после чего она становится доступной всем участникам проекта. На рисунке 2.3.13 представлен процесс создания kanban-доски.

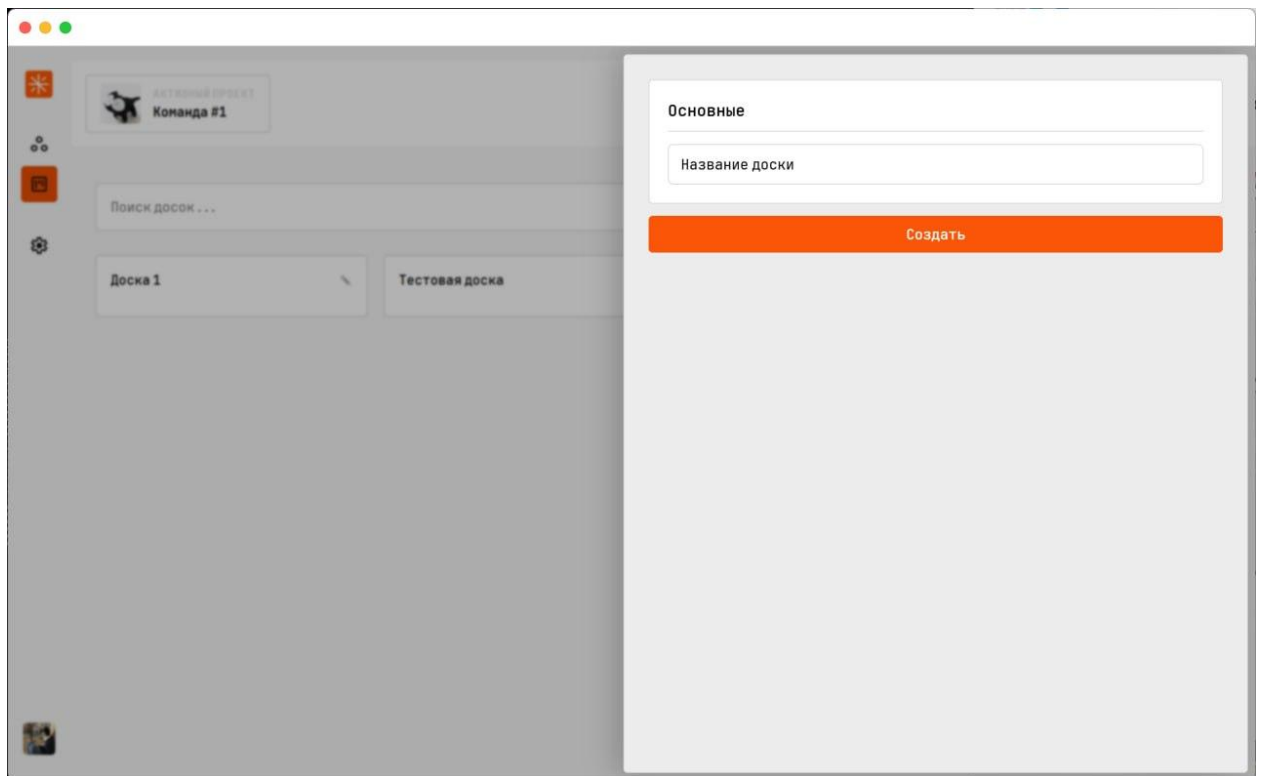


Рисунок 2.3.13 - Форма создания канбан-досок

На рисунке 2.3.14 представлен фрагмент программного кода, отвечающий за создание новой доски.

```
func (s *Service) CreateBoard(workspaceID string, name string) (*pb2.Board, error) {
    if s.state.Token == "" {
        return nil, fmt.Errorf("auth required")
    }
    return s.grpc.Kanban.CreateBoard(context.Background(), &pb2.CreateBoardRequest{
        WorkspaceId: workspaceID, Name: name,
    })
}
```

Рисунок 2.3.14 - Фрагмент кода создания доски

После открытия доски отображается рабочая область, содержащая колонки и карточки задач. На рисунке 2.3.15 представлен интерфейс kanban-доски.

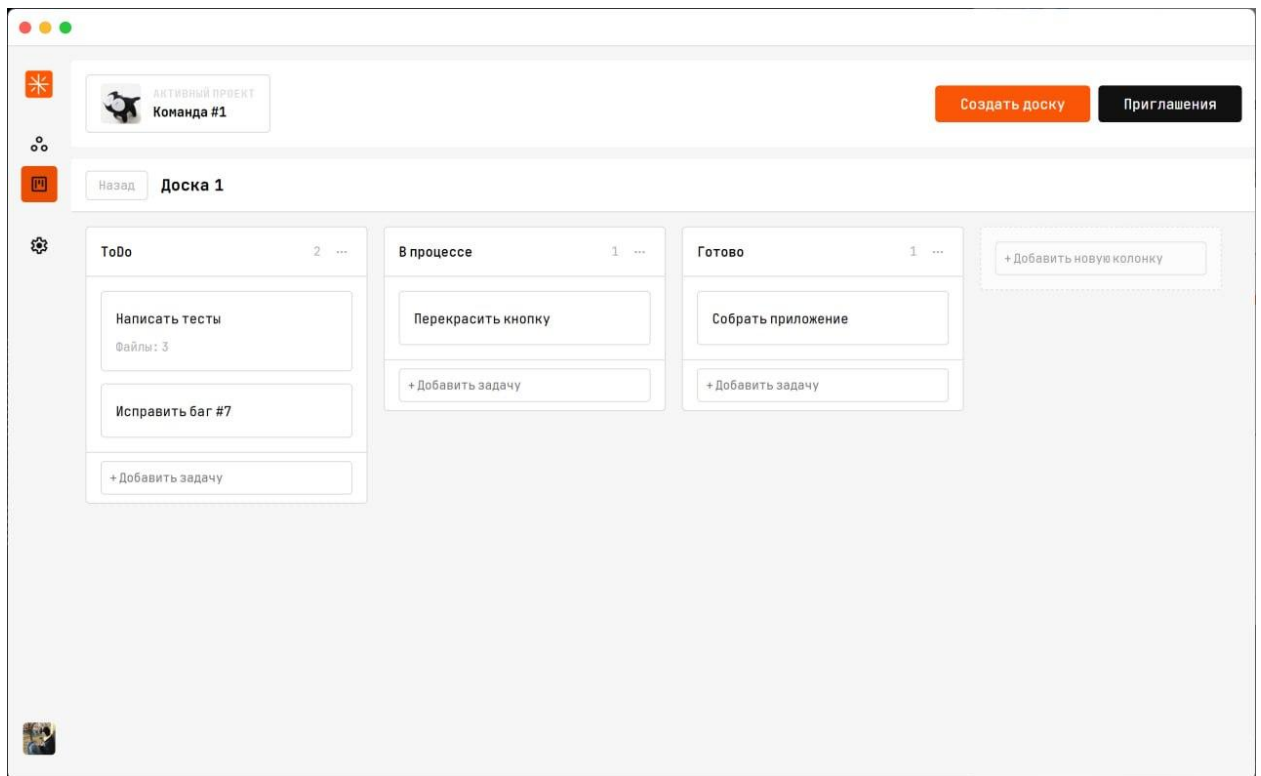


Рисунок 2.3.15 - интерфейс канбан-доски

На рисунке 2.3.16 представлен фрагмент программного кода, реализующий создание колонки.

```
func (s *Service) CreateColumn(boardID string, name string, sortKey string) (*pb2.BoardColumn, error) {
    if s.state.Token == "" {
        return nil, fmt.Errorf("auth required")
    }
    return s.grpc.KanbanColumn.CreateColumn(context.Background(), &pb2.CreateColumnRequest{
        BoardId: boardID, Name: name, SortKey: sortKey,
    })
}
```

Рисунок 2.3.16 - Фрагмент кода создания колонок

На рисунке 2.3.17 представлен фрагмент программного кода, реализующий создание задачи.

```

func (s *Service) CreateTask(workspaceID, boardID, columnID, title, description, priority string, startDate,
dueDate, parentTaskID *string, sortKey string) (*pb2.Task, error) {
    if s.state.Token == "" {
        return nil, fmt.Errorf("auth required")
    }

    var p pb2.TaskPriority
    var enumKey string
    switch priority {
    case "low":
        enumKey = "TASK_PRIORITY_LOW"
    case "medium":
        enumKey = "TASK_PRIORITY_MEDIUM"
    case "high":
        enumKey = "TASK_PRIORITY_HIGH"
    case "critical":
        enumKey = "TASK_PRIORITY_CRITICAL"
    default:
        enumKey = "TASK_PRIORITY_MEDIUM"
    }
    if val, ok := pb2.TaskPriority_value[enumKey]; ok {
        p = pb2.TaskPriority(val)
    } else {
        p = pb2.TaskPriority(pb2.TaskPriority_value["TASK_PRIORITY_MEDIUM"])
    }

    req := &pb2.CreateTaskRequest{
        WorkspaceId: workspaceID,
        BoardId:     boardID,
        ColumnId:    columnID,
        Title:       title,
        Description: description,
        Priority:     p,
        SortKey:     sortKey,
    }
    if dueDate != nil {
        req.DueDate = dueDate
    }
    if parentTaskID != nil && *parentTaskID != "" {
        req.ParentTaskId = parentTaskID
    }

    return s.grpc.Task.CreateTask(context.Background(), req)
}

```

Рисунок 2.3.17 - Фрагмент кода создания задач

Каждая задача содержит основные сведения о выполняемой работе, включая название, описание, приоритет и срок выполнения. На рисунке 2.3.18 представлено окно просмотра задачи.

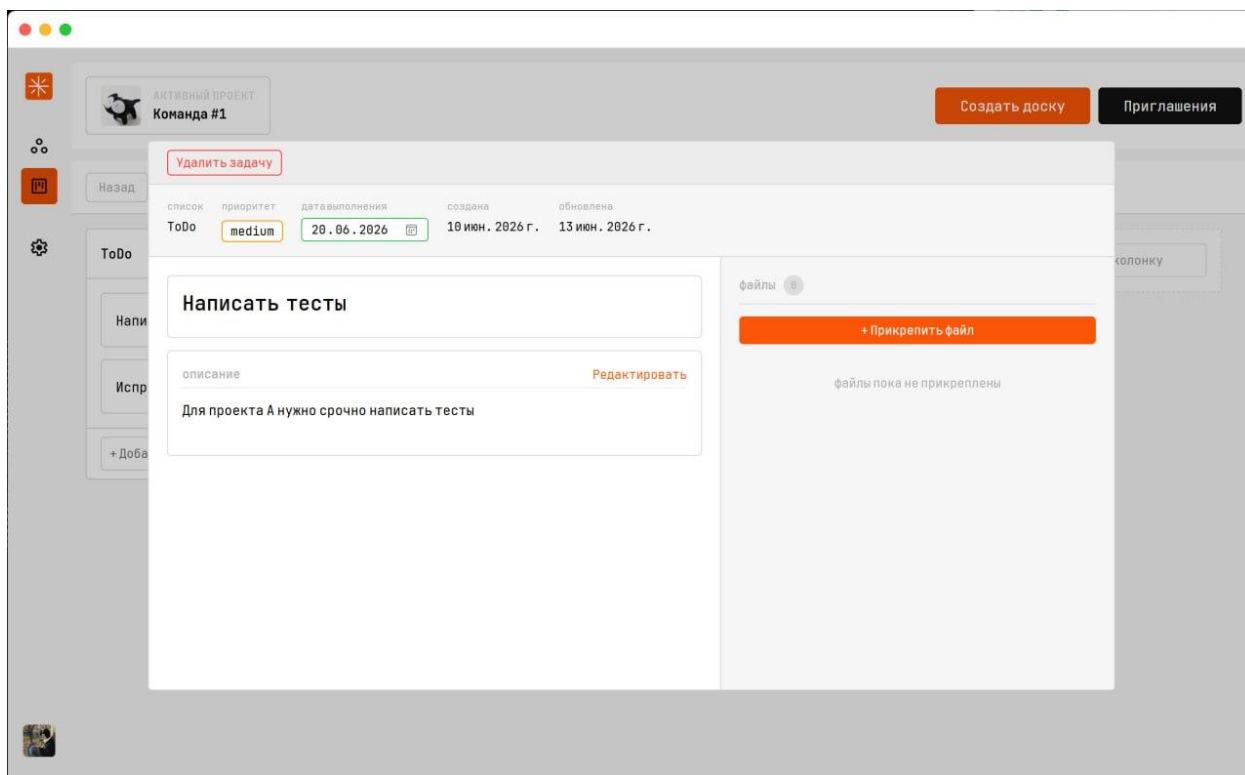


Рисунок 2.3.18 - Окно просмотра задачи

Одним из наиболее важных элементов интерфейса является механизм перемещения задач между колонками. Пользователь может изменять состояние задачи путем её перетаскивания из одной колонки в другую.

Использование технологии Drag-and-Drop позволяет значительно упростить работу с задачами и обеспечивает наглядное отображение текущего состояния проекта.

Для хранения дополнительной информации реализован механизм прикрепления файлов к задачам. На рисунке 2.3.20 представлен интерфейс работы с вложениями.

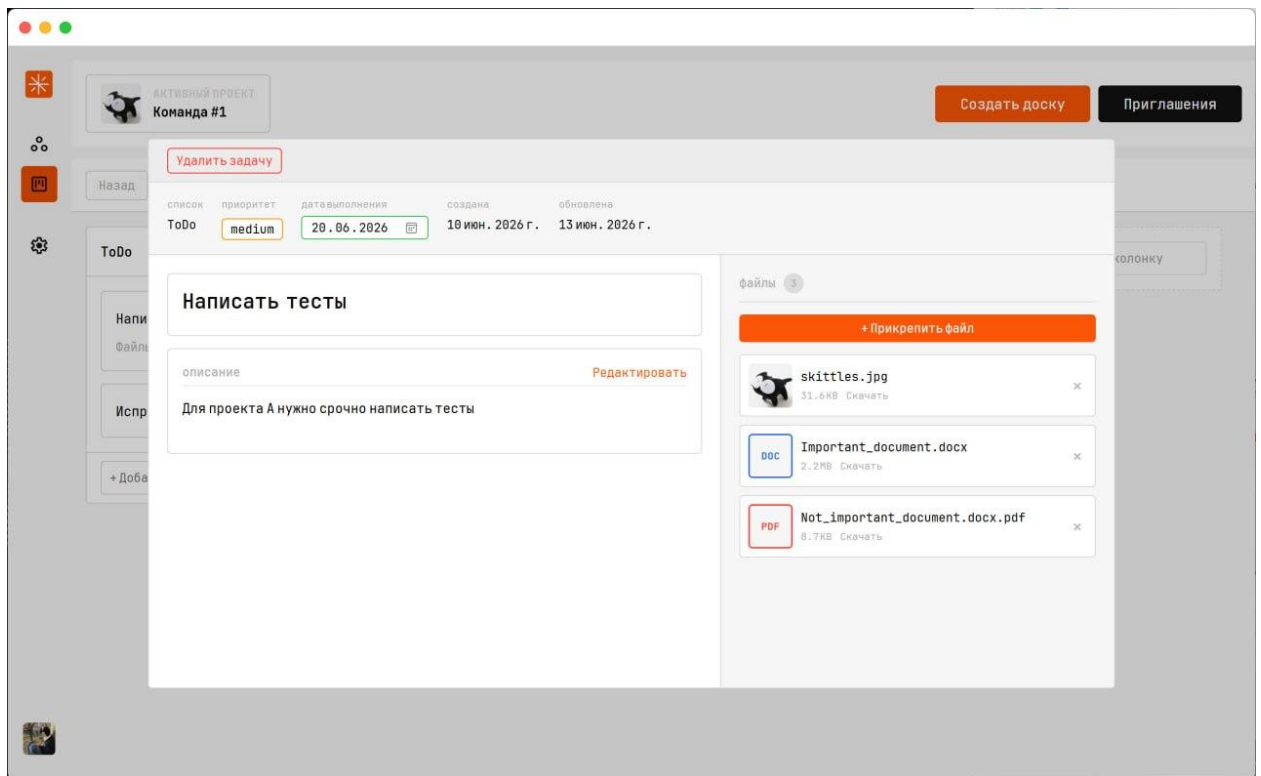


Рисунок 2.3.20 - Интерфейс вложений

На рисунке 2.3.21 представлен фрагмент программного кода, реализующий загрузку вложений.

```
func (s *Service) AddAttachment(taskID, fileName, fileURL, fileType string, fileSize int64) (*pb2.TaskAttachment, error) {
    ctx, err := s.authContext()
    if err != nil {
        return nil, err
    }

    return s.grpc.TaskAttachment.AddAttachment(ctx, &pb2.AddAttachmentRequest{
        TaskId: taskID,
        FileName: fileName,
        FileUrl: fileURL,
        FileType: fileType,
        FileSize: fileSize,
    })
}
```

Рисунок 2.3.21 - Фрагмент кода загрузки вложений

Дополнительно в приложении реализован раздел профиля пользователя, предназначенный для просмотра и изменения персональных данных учетной записи. На рисунке 2.3.22 представлен профиль пользователя.

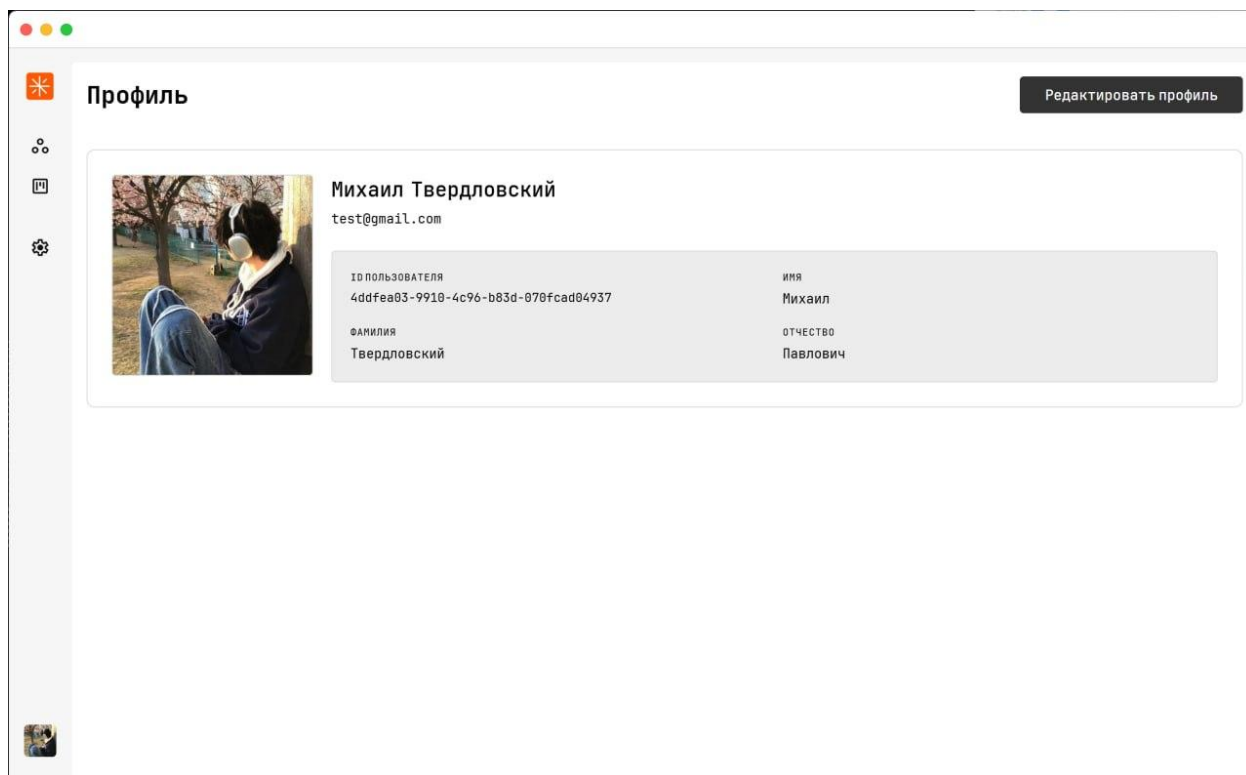


Рисунок 2.3.22 - Страница профиля

На рисунке 2.3.23 представлен фрагмент программного кода, реализующий обновление профиля пользователя.



```
func (s *Service) UpdateProfile(input UpdateProfileInput,) (*pb2.User, error) {
    if s.state.CurrentUser == nil {
        return nil, fmt.Errorf("user not authenticated")
    }

    req := &pb2.UpdateProfileRequest{
        UserId: s.state.CurrentUser.UserId,
        User: &pb2.User{
            FirstName: input.FirstName,
            LastName:  input.LastName,
        },
    }

    if input.MiddleName != "" {
        req.User.MiddleName = &input.MiddleName
    }

    if input.PhotoBase64 != "" {
        photoBytes, err := base64.StdEncoding.DecodeString(
            input.PhotoBase64,
        )

        if err != nil {
            return nil, fmt.Errorf(
                "failed to decode photo: %v",
                err,
            )
        }

        req.PhotoData = photoBytes
        req.PhotoExtension = input.PhotoExtension
    }

    resp, err := s.grpc.User.UpdateProfile(
        context.Background(),
        req,
    )

    if err != nil {
        return nil, fmt.Errorf(
            "grpc update profile error: %v",
            err,
        )
    }

    if resp.Success {
        return s.refreshCurrentProfile()
    }

    return nil, fmt.Errorf("failed to update profile")
}
```

Рисунок 2.3.23 - Фрагмент кода обновления профиля

Для персонализации работы с приложением реализован раздел настроек. На рисунке 2.3.24 представлен раздел настроек приложения.

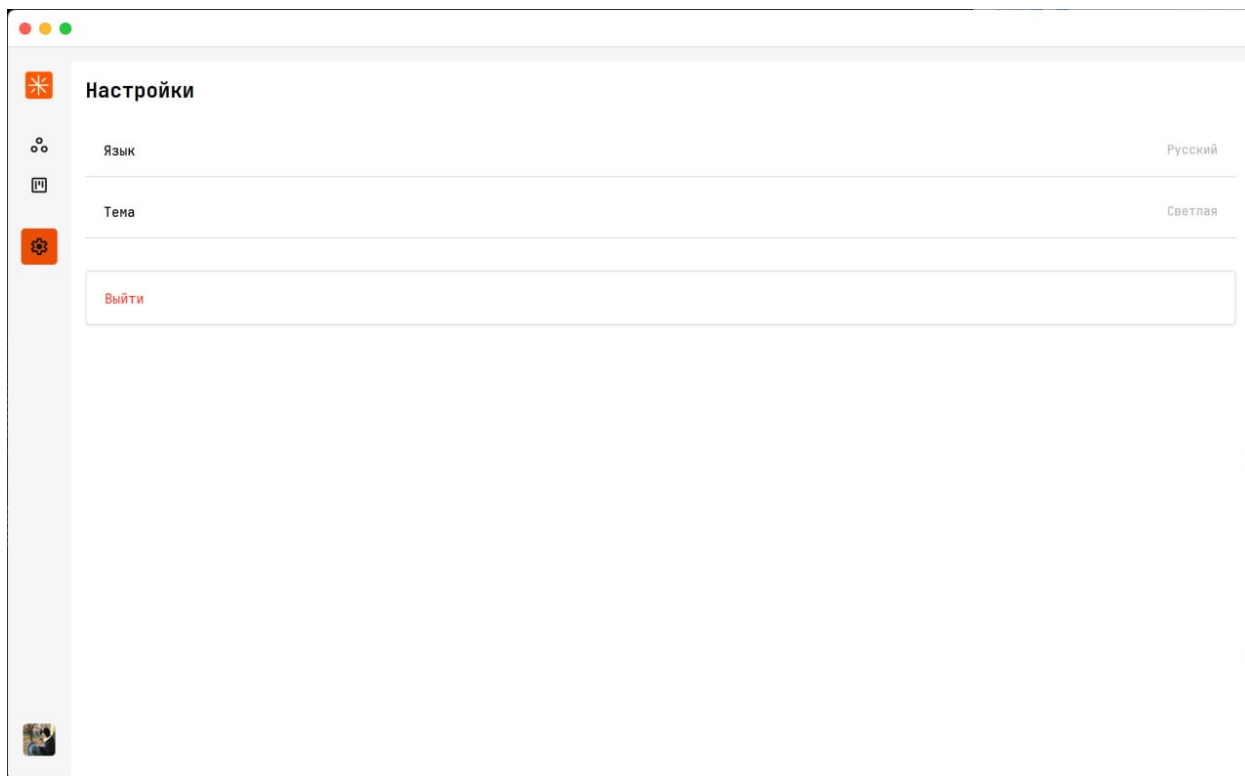


Рисунок 2.3.24 - Страница профиля

Важной особенностью клиентской части является поддержка обновления данных в режиме реального времени с использованием gRPC Streaming [4]. При изменении задач, рабочих пространств или досок информация автоматически обновляется у всех подключенных пользователей.

Благодаря использованию потоковой передачи данных пользователи получают актуальную информацию без необходимости ручного обновления интерфейса.

2.3.1 Руководство пользователя

Информационная система “Atlas” предназначена для организации командной работы, управления рабочими пространствами и контроля

выполнения задач. Приложение предоставляет пользователям инструменты для создания рабочих пространств, взаимодействия с участниками проектов, работы с kanban-досками и отслеживания хода выполнения задач.

Для начала работы необходимо запустить приложение “Atlas”. После запуска пользователю отображается окно авторизации, представленное на рисунке 2.3.1. Для входа в систему необходимо указать адрес электронной почты и пароль либо воспользоваться авторизацией через учетную запись Google. При отсутствии учетной записи пользователь может перейти к форме регистрации и создать новый аккаунт.

После успешной авторизации открывается главное окно приложения, представленное на рисунке 2.3.3. Навигация между основными разделами системы осуществляется с помощью боковой панели навигации, обеспечивающей быстрый доступ к рабочим пространствам, задачам, профилю пользователя и настройкам приложения.

Основным разделом приложения является раздел “Рабочие пространства”. Рабочее пространство представляет собой отдельную область для организации работы над проектом и хранения связанных с ним данных. В разделе отображается список доступных пользователю рабочих пространств с краткой информацией о каждом проекте.

Для создания нового рабочего пространства необходимо воспользоваться формой создания проекта, представленной на рисунке 2.3.4. В процессе создания пользователь указывает название и описание проекта, выбирает тип рабочего пространства (персональное или командное), а также при необходимости может пригласить участников проекта. После подтверждения операции новое рабочее пространство появляется в общем списке доступных проектов.

Для управления параметрами рабочего пространства используется окно настроек рабочего пространства, представленное на рисунке 2.3.7. В данном разделе отображается информация о проекте, его участниках и владельце.

Пользователь, обладающий необходимыми правами доступа, может изменять сведения о рабочем пространстве.

Для организации совместной работы реализован механизм приглашений. Интерфейс отправки приглашений представлен на рисунке 2.3.8. Для приглашения нового участника необходимо указать адрес его электронной почты и выбрать роль, которая будет назначена пользователю после вступления в рабочее пространство. Полученные приглашения отображаются в отдельном разделе приложения, где пользователь может принять или отклонить приглашение.

После принятия приглашения пользователь становится участником рабочего пространства и получает доступ к его содержимому в соответствии с назначенной ролью. Список участников проекта представлен на рисунке 2.3.12.

Для управления рабочими процессами внутри рабочего пространства используются kanban-доски. Список доступных досок отображается в соответствующем разделе рабочего пространства, представленном на рисунке 2.3.13. Для создания новой доски необходимо открыть форму создания, показанную на рисунке 2.3.14, указать название доски и подтвердить операцию.

После открытия доски пользователю становится доступна рабочая область, представленная на рисунке 2.3.16. Доска состоит из набора колонок, каждая из которых соответствует определённому этапу выполнения работ. Пользователь может создавать новые колонки и изменять их названия в соответствии с особенностями рабочего процесса проекта.

Внутри колонок размещаются карточки задач. Для создания новой задачи необходимо выбрать нужную колонку и заполнить форму создания задачи. Каждая задача содержит основную информацию о выполняемой работе, включая название, описание, приоритет и срок выполнения.

Для просмотра и редактирования информации используется окно задачи, представленное на рисунке 2.3.19. В карточке задачи пользователь

может изменять параметры задачи, отслеживать ход выполнения работы и просматривать связанную информацию.

Изменение состояния задачи осуществляется посредством её перемещения между колонками kanban-доски. Для этого используется механизм Drag-and-Drop, позволяющий перетаскивать карточки задач между этапами выполнения работ. После перемещения изменения автоматически сохраняются и становятся доступны другим участникам проекта.

Для хранения дополнительных материалов предусмотрен механизм вложений. Интерфейс работы с вложениями представлен на рисунке 2.3.20. Пользователь может прикреплять к задачам документы, изображения и другие файлы, необходимые для выполнения работы.

Для управления персональными данными используется раздел “Профиль”, представленный на рисунке 2.3.22. В данном разделе доступны просмотр и изменение сведений учетной записи.

Для персонализации работы приложения предусмотрен раздел настроек. В настройках пользователь может выбрать светлую или тёмную тему оформления интерфейса, изменить язык приложения, а также выполнить выход из учетной записи.

Одной из особенностей системы является поддержка синхронизации данных в режиме реального времени. Все изменения, связанные с рабочими пространствами, участниками проектов, досками и задачами, автоматически отображаются у всех пользователей, имеющих доступ к соответствующему проекту. Благодаря этому обеспечивается актуальность информации и повышается эффективность совместной работы.

Таким образом, разработанная информационная система “Atlas” предоставляет пользователям удобный набор инструментов для организации проектной деятельности, управления задачами и взаимодействия участников команды в едином информационном пространстве.

2.4 Разработка серверной части

Серверная часть информационной системы “Atlas” реализована на языке программирования Go [1] и предназначена для хранения данных, обработки пользовательских запросов, управления проектами и обеспечения взаимодействия между участниками системы. В качестве основного протокола взаимодействия между клиентской и серверной частью используется gRPC [4], обеспечивающий высокую производительность обмена данными и строгую типизацию передаваемых сообщений благодаря использованию Protocol Buffers [6].

Основной задачей серверной части является обработка всех операций, выполняемых пользователями в системе. К таким операциям относятся регистрация и авторизация пользователей, создание рабочих пространств, управление участниками проектов, создание kanban-досок и задач, а также синхронизация данных между пользователями в режиме реального времени.

После регистрации нового пользователя сервер выполняет проверку корректности введенных данных и создает новую учетную запись в базе данных. Для защиты пользовательских данных пароли не сохраняются в открытом виде. Перед записью в базу данных пароль проходит процедуру хеширования с использованием алгоритма bcrypt [15]. На рисунке 2.4.2 представлен фрагмент программного кода, реализующий хеширование пароля пользователя.

A code editor window with a white background and a title bar with three colored circles (red, yellow, green). It contains Go code for password hashing and checking using the bcrypt library.

```
package pkg

import "golang.org/x/crypto/bcrypt"

func HashPassword(p string) (string, error) {
    h, err := bcrypt.GenerateFromPassword([]byte(p), bcrypt.DefaultCost)
    return string(h), err
}

func CheckPassword(p, h string) bool {
    if h == "" {
        return false
    }
    return bcrypt.CompareHashAndPassword([]byte(h), []byte(p)) == nil
}
```

Рисунок 2.4.4 - Хеширование и проверки пароля

После успешной авторизации сервер генерирует JWT-токен [14], содержащий идентификатор пользователя и срок действия сессии. Полученный токен передается клиентскому приложению и используется для последующей авторизации запросов. На рисунке 2.4.3 представлен фрагмент программного кода генерации JWT-токена.

```

package auth

import (
    "errors"
    "github.com/golang-jwt/jwt/v5"
    "github.com/google/uuid"
    "os"
    "time"
)

func getJWTSecret() []byte {
    return []byte(os.Getenv("JWT_SECRET"))
}

func GenerateToken(userID uuid.UUID) (string, error) {
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
        "sub": userID.String(),
        "exp": time.Now().Add(time.Hour * 72).Unix(),
    })
    return token.SignedString(getJWTSecret())
}

func ParseToken(tokenString string) (string, error) {
    token, err := jwt.Parse(tokenString, func(token *jwt.Token) (interface{}, error) {
        return getJWTSecret(), nil
    })
    if err != nil || !token.Valid {
        return "", err
    }
    claims, ok := token.Claims.(jwt.MapClaims)
    if !ok || !token.Valid {
        return "", errors.New("invalid token")
    }
    return claims["sub"].(string), nil
}

```

Рисунок 2.4.5 - Функции генерации JWT-токена и парсинга JWT-токена

Для защиты серверных методов реализован механизм проверки токенов. При поступлении каждого запроса выполняется проверка подписи токена, срока его действия и принадлежности пользователю. После успешной проверки идентификатор пользователя добавляется в контекст запроса и становится доступным для дальнейшей обработки.

Для хранения информации о пользователях, проектах, досках и задачах используется СУБД PostgreSQL [7]. В базе данных реализованы таблицы пользователей, рабочих пространств, участников проектов, приглашений, досок, колонок, задач и файловых вложений. На рисунке 2.4.5 представлена структура базы данных приложения.

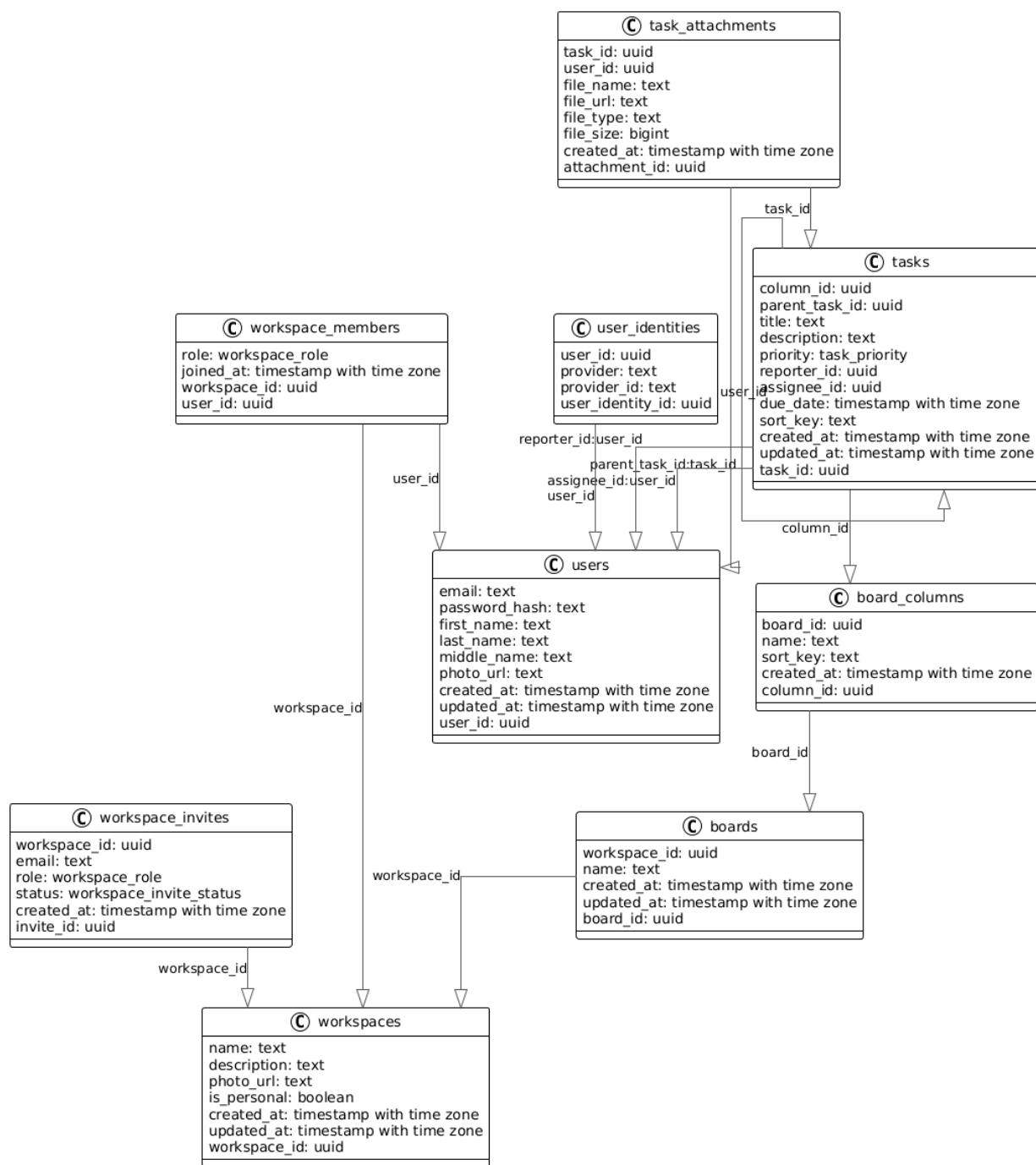


Рисунок 2.4.5 - Структура базы данных

После создания пользователем нового рабочего пространства сервер выполняет проверку введенных данных и создает соответствующую запись в базе данных. Одновременно создается связь между пользователем и рабочим пространством, а пользователю автоматически назначается роль владельца проекта. На рисунке 2.4.6 представлен фрагмент программного кода создания рабочего пространства.

```

func (s *WorkspaceService) CreateWorkspace(ctx context.Context, req *proto.CreateWorkspaceRequest, userID uuid.UUID)
(*proto.Workspace, error) {
    if userID == uuid.Nil {
        return nil, pkg.ErrUnauthenticated
    }

    photoPath := "/static/workspaces/ws_default_image.jpg"
    inputPhoto := req.GetPhotoUrl()

    if inputPhoto != "" {
        var newFile string
        var err error

        if strings.HasPrefix(inputPhoto, "data:image") {
            newFile, err = pkg.SaveImageResource(inputPhoto, "./static/workspaces")
        } else if strings.HasPrefix(inputPhoto, "http") && !strings.HasPrefix(inputPhoto, "blob:") {
            newFile, err = pkg.DownloadPhoto(inputPhoto, "./static/workspaces")
        }

        if err == nil && newFile != "" {
            photoPath = "/static/workspaces/" + newFile
        }
    }

    ws := &domain.Workspace{
        WorkspaceName: req.Name,
        WorkspaceDescription: req.Description,
        WorkspacePhoto: photoPath,
        IsPersonal: req.IsPersonal,
    }

    created, err := s.repo.CreateWorkspace(ctx, ws, userID)

    fullWs, _ := s.repo.GetWorkspaceByID(ctx, created.WorkspaceID)
    s.hub.SubscribeClientToWorkspace(userID, created.WorkspaceID.String())
    protoWs := mapDomainToProto(fullWs)
    s.hub.BroadcastWorkspaceCreated(protoWs.WorkspaceID, protoWs, uuid.Nil)

    return protoWs, nil
}

```

Рисунок 2.4.6 - Фрагмент программного кода создания рабочего пространства

Для организации совместной работы реализован механизм приглашений. Пользователь с ролью владельца или администратора может отправить приглашение другому сотруднику. После получения запроса сервер проверяет существование пользователя, формирует приглашение и сохраняет его в базе данных. На рисунке 2.4.7 представлен фрагмент программного кода создания приглашения.


```
func (s *WorkspaceInviteService) CreateInvite(ctx context.Context, req *proto.CreateInviteRequest, senderID
uuid.UUID) (*proto.WorkspaceInvite, error) {
    workspaceID, err := uuid.Parse(req.WorkspaceId)
    if err != nil {
        return nil, pkg.ErrInvalidArgument
    }
    email := strings.ToLower(
        strings.TrimSpace(req.Email),
    )
    role, err := s.workspaceRepo.GetMemberRole(ctx, workspaceID, senderID)
    if role != "owner" && role != "admin" {
        return nil, pkg.ErrForbidden
    }
    sender, err := s.userRepo.GetByID(ctx, senderID)
    if strings.ToLower(sender.Email) == email {
        return nil, fmt.Errorf("cannot invite yourself")
    }
    userExists, err := s.userRepo.ExistsByEmail(ctx, email)
    if !userExists {
        return nil, fmt.Errorf("user not found")
    }
    hasInvite, err := s.repo.HasPendingInvite(ctx, workspaceID, email)
    if hasInvite {
        return nil, fmt.Errorf("invite already exists")
    }
    user, err := s.userRepo.GetByEmail(ctx, email)
    isMember, err := s.workspaceRepo.IsMember(
        ctx,
        workspaceID,
        user.UserID,
    )
    roleString := "member"
    switch req.Role {
    case proto.WorkspaceRole_WORKSPACE_ROLE_ADMIN:
        roleString = "admin"
    case proto.WorkspaceRole_WORKSPACE_ROLE_MEMBER:
        roleString = "member"
    }
    invite, err := s.repo.CreateInvite(ctx, workspaceID, email, roleString)
    if err != nil {
        return nil, err
    }
    protoInvite := mapInviteToProto(invite)
    s.hub.BroadcastInviteCreated(req.WorkspaceId, protoInvite, senderID)
    return protoInvite, nil
}
```

Рисунок 2.4.7 - Фрагмент программного кода создания приглашения

После подтверждения приглашения сервер автоматически добавляет пользователя в состав участников рабочего пространства и назначает ему выбранную роль доступа. На рисунке 2.4.8 представлен фрагмент программного кода обработки принятия приглашения.

```
func (s *WorkspaceInviteService) RespondToInvite(ctx context.Context, inviteID uuid.UUID, accept bool, userID
uuid.UUID) error {
    invite, err := s.repo.GetInviteByID(ctx, inviteID)
    if err != nil {
        return err
    }
    if invite.Status != "pending" {
        return fmt.Errorf("invite already handled")
    }
    user, err := s.userRepo.GetByID(ctx, userID)
    if err != nil {
        return err
    }
    if strings.ToLower(user.Email) != strings.ToLower(invite.Email) {
        return pkg.ErrForbidden
    }
    if !accept {
        err = s.repo.UpdateInviteStatus(ctx, inviteID, "rejected")
        if err != nil {
            return err
        }
    }

    return nil
}

isMember, err := s.workspaceRepo.IsMember(ctx, invite.WorkspaceID, userID)
if err != nil {
    return err
}
err = s.workspaceRepo.JoinWorkspace(ctx, invite.WorkspaceID, userID)
if err != nil {
    return err
}
err = s.repo.UpdateInviteStatus(ctx, inviteID, "accepted")

if err != nil {
    return err
}
err = s.repo.DeletePendingInvites(ctx, invite.WorkspaceID, invite.Email)
if err != nil {
    s.logger.Warn(
        "failed to cleanup pending invites",
        zap.Error(err),
    )
}
s.hub.SubscribeClientToWorkspace(userID, invite.WorkspaceID.String())
return nil
}
```

Рисунок 2.4.8 - Фрагмент программного кода обработки принятия приглашения

Внутри каждого рабочего пространства пользователи могут создавать kanban-доски [25]. При создании доски сервер формирует новую запись в базе данных и связывает её с выбранным рабочим пространством. На рисунке 2.4.9 представлен фрагмент программного кода создания kanban-доски.

```

func (s *KanbanService) CreateBoard(ctx context.Context, req *proto.CreateBoardRequest) (*proto.Board, error) {
    wsID, _ := uuid.Parse(req.WorkspaceId)
    senderID, _ := middleware.GetUserIDFromContext(ctx)
    role, err := s.repo.GetWorkspaceRole(ctx, wsID, senderID)
    if err != nil {
        return nil, pkg.ErrForbidden
    }
    if !s.canEdit(role) {
        return nil, pkg.ErrForbidden
    }
    board, err := s.repo.CreateBoard(ctx, &domain.Board{WorkspaceID: wsID, Name: req.Name})
    if err != nil {
        s.logger.Error("failed to create board", zap.Error(err))
        return nil, err
    }
    res := mapBoardToProto(board)
    s.hub.BroadcastBoardCreated(req.WorkspaceId, res, uuid.Nil)
    return res, nil
}

```

Рисунок 2.4.9 - Фрагмент программного кода создания kanban-доски

Каждая доска содержит набор колонок и карточек задач. При создании новой колонки или задачи сервер выполняет проверку прав пользователя, сохраняет данные в базе и возвращает обновленную информацию клиентскому приложению. На рисунке 2.4.10 представлен фрагмент программного кода создания колонки.

```

func (s *KanbanService) CreateColumn(ctx context.Context, req *proto.CreateColumnRequest) (*proto.BoardColumn, error) {
    bID, _ := uuid.Parse(req.BoardId)
    senderID, _ := middleware.GetUserIDFromContext(ctx)

    wsID, err := s.repo.GetBoardWorkspaceID(ctx, bID)
    if err != nil {
        return nil, err
    }
    role, err := s.repo.GetWorkspaceRole(ctx, wsID, senderID)
    if err != nil {
        return nil, pkg.ErrForbidden
    }
    if !s.canEdit(role) {
        return nil, pkg.ErrForbidden
    }
    col, err := s.repo.CreateColumn(ctx, &domain.BoardColumn{
        BoardID: bID,
        Name:    req.Name,
    })
    if err != nil {
        s.logger.Error("failed to create column", zap.Error(err))
        return nil, err
    }
    res := mapColumnToProto(col)
    s.hub.BroadcastColumnCreated(wsID.String(), res, uuid.Nil)
    return res, nil
}

```

Рисунок 2.4.10 - Фрагмент программного кода создания колонки

На рисунке 2.4.11 представлен фрагмент программного кода создания задачи.

```
func (s *TaskService) CreateTask(ctx context.Context, req *proto.CreateTaskRequest, userID uuid.UUID) (*proto.Task, error) {
    workspaceID, err := uuid.Parse(req.WorkspaceId)
    if err != nil {
        return nil, pkg.ErrInvalidArgument
    }
    boardID, err := uuid.Parse(req.BoardId)
    if err != nil {
        return nil, pkg.ErrInvalidArgument
    }
    columnID, err := uuid.Parse(req.ColumnId)
    if err != nil {
        return nil, pkg.ErrInvalidArgument
    }

    if err := s.checkMember(ctx, workspaceID, userID); err != nil {
        return nil, err
    }
    t := &domain.Task{
        WorkspaceID: workspaceID,
        BoardID:     boardID,
        ColumnID:    columnID,
        Title:       req.Title,
        Description: req.Description,
        Priority:     domain.TaskPriority(req.Priority),
        ReporterID:  userID,
        SortKey:     req.SortKey,
    }
    if req.ParentTaskId != nil {
        id, err := uuid.Parse(*req.ParentTaskId)
        if err != nil {
            return nil, pkg.ErrInvalidArgument
        }
        t.ParentTaskID = &id
    }
    if req.AssigneeId != nil {
        id, err := uuid.Parse(*req.AssigneeId)
        if err != nil {
            return nil, pkg.ErrInvalidArgument
        }
        t.AssigneeID = &id
    }
    if req.DueDate != nil {
        parsedTime, err := time.Parse(time.RFC3339, *req.DueDate)
        if err == nil {
            t.DueDate = &parsedTime
        }
    }
    created, err := s.repo.Create(ctx, t)
    if err != nil {
        return nil, err
    }
    res := s.mapTaskToProto(created, &domain.TaskCounts{})
    s.hub.BroadcastTaskCreated(req.WorkspaceId, res, userID)
    return res, nil
}
```

Рисунок 2.4.11 - Фрагмент программного кода создания колонки

Для обеспечения контроля выполнения работ каждая задача может содержать описание, срок выполнения, приоритет и прикрепленные файлы.

Все изменения, вносимые пользователями в карточки задач, сохраняются сервером в базе данных и становятся доступными другим участникам проекта.

Для хранения файлов реализован отдельный модуль обработки вложений. После получения файла сервер выполняет его проверку, сохраняет файл на диске и формирует ссылку для дальнейшего доступа к нему. На рисунке 2.4.12 представлен фрагмент программного кода загрузки вложений.

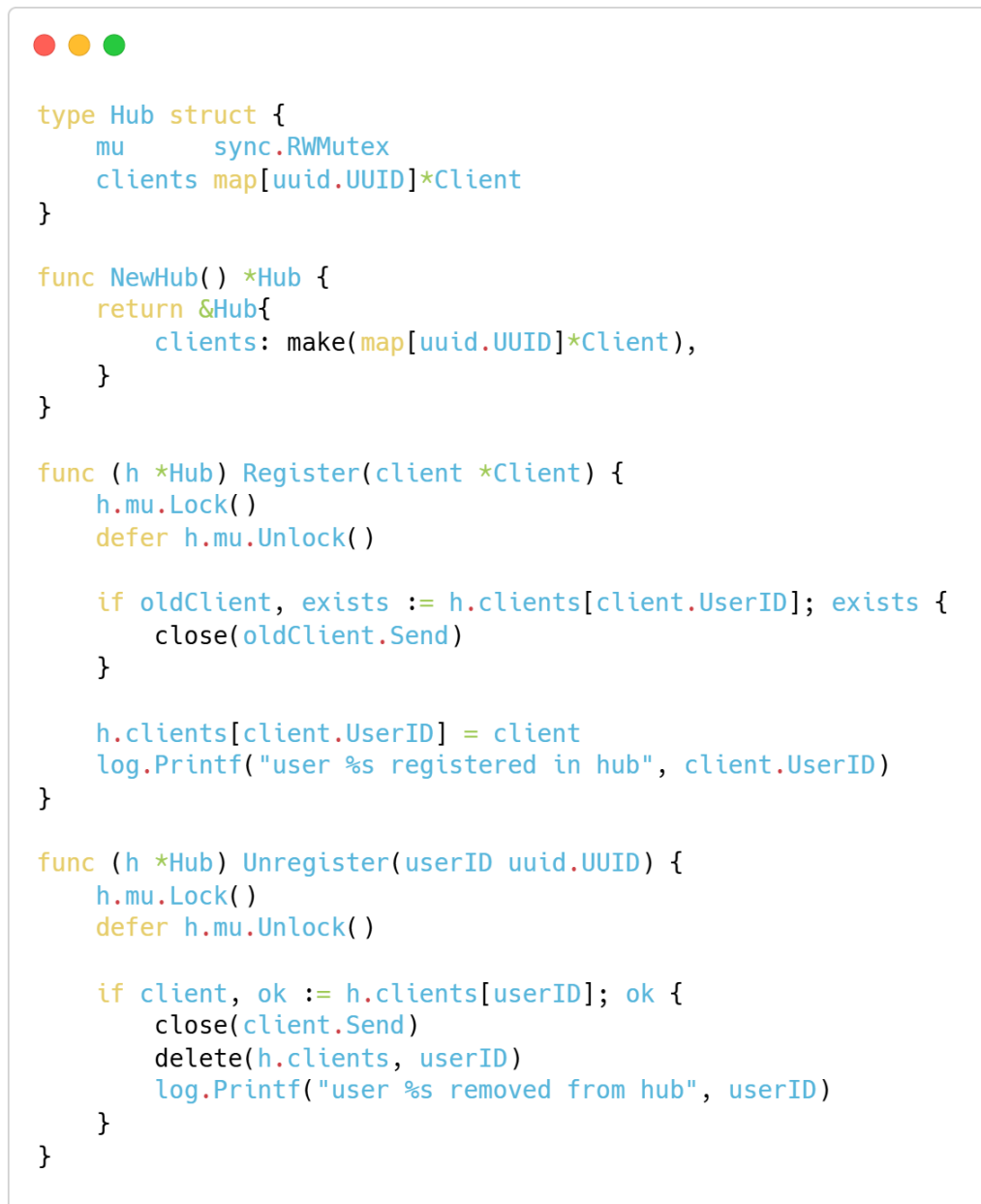
```
func (s *TaskService) AddAttachment(ctx context.Context, req *proto.AddAttachmentRequest, userID uuid.UUID)
(*proto.TaskAttachment, error) {
    taskID, err := uuid.Parse(req.TaskId)
    if err != nil {
        return nil, pkg.ErrInvalidArgument
    }
    task, err := s.repo.GetByID(ctx, taskID)
    if err != nil {
        return nil, err
    }
    if err := s.checkMember(ctx, task.WorkspaceID, userID); err != nil {
        return nil, err
    }
    savedName, downloadedSize, err := pkg.DownloadAttachment(req.FileUrl, "static/files", req.FileName)
    if err != nil {
        return nil, fmt.Errorf("failed to download attachment: %w", err)
    }
    localURL := "/static/files/" + savedName
    a := &domain.TaskAttachment{
        TaskID:    taskID,
        UserID:    userID,
        FileName:  req.FileName,
        FileURL:   localURL,
        FileType:  req.FileType,
        FileSize:  downloadedSize,
    }
    created, err := s.repo.AddAttachment(ctx, a)
    res := s.mapAttachmentToProto(created)
    fileEvent := &proto.FileAttachment{
        FileId: res.AttachmentId,
        Name:   res.FileName,
        Url:    res.FileUrl,
    }
    s.hub.BroadcastAttachmentAdded(task.WorkspaceID.String(), taskID.String(), fileEvent, userID)
    return res, nil
}
```

Рисунок 2.4.12 - Фрагмент программного кода загрузки вложений

Одной из наиболее важных функций серверной части является поддержка совместной работы пользователей в режиме реального времени. Для реализации данного механизма используются потоковые gRPC-соединения (gRPC Streaming) [5].

После подключения клиента сервер регистрирует его в системе обмена событиями и поддерживает постоянное соединение для передачи изменений.

При создании задачи, изменении доски или обновлении рабочего пространства сервер автоматически формирует событие и отправляет его всем участникам соответствующего проекта. На рисунке 2.4.13 представлен фрагмент программного кода регистрации клиента в системе потоковых событий.



```
type Hub struct {
    mu      sync.RWMutex
    clients map[uuid.UUID]*Client
}

func NewHub() *Hub {
    return &Hub{
        clients: make(map[uuid.UUID]*Client),
    }
}

func (h *Hub) Register(client *Client) {
    h.mu.Lock()
    defer h.mu.Unlock()

    if oldClient, exists := h.clients[client.UserID]; exists {
        close(oldClient.Send)
    }

    h.clients[client.UserID] = client
    log.Printf("user %s registered in hub", client.UserID)
}

func (h *Hub) Unregister(userID uuid.UUID) {
    h.mu.Lock()
    defer h.mu.Unlock()

    if client, ok := h.clients[userID]; ok {
        close(client.Send)
        delete(h.clients, userID)
        log.Printf("user %s removed from hub", userID)
    }
}
```

Рисунок 2.4.13 - Фрагмент программного кода регистрации клиента в системе потоковых событий

На рисунке 2.4.14 представлен фрагмент программного кода отправки событий пользователям.



```
func (h *Hub) BroadcastPresence(workspaceID string, userID string, status proto.UserStatus, excludeID uuid.UUID) {
    h.BroadcastToWorkspace(workspaceID, &proto.ServerStreamEvent{
        Event: &proto.ServerStreamEvent_Presence{
            Presence: &proto.UserPresence{UserId: userID, Status: status},
        },
    }, excludeID)
}

func (h *Hub) BroadcastProfileUpdated(workspaceID string, user *proto.User, excludeID uuid.UUID) {
    h.BroadcastToWorkspace(workspaceID, &proto.ServerStreamEvent{
        Event: &proto.ServerStreamEvent_Profile{
            Profile: &proto.ProfileUpdated{User: user},
        },
    }, excludeID)
}

func (h *Hub) BroadcastAvatarChanged(workspaceID string, userID string, newPhotoURL string, excludeID uuid.UUID) {
    h.BroadcastToWorkspace(workspaceID, &proto.ServerStreamEvent{
        Event: &proto.ServerStreamEvent_Avatar{
            Avatar: &proto.AvatarChanged{UserId: userID, NewPhotoUrl: newPhotoURL},
        },
    }, excludeID)
}
```

Рисунок 2.4.14 - Фрагмент программного кода отправки событий пользователям

Использование потоковой передачи данных позволяет всем участникам проекта получать актуальную информацию без необходимости ручного обновления интерфейса. Благодаря этому руководитель проекта может оперативно отслеживать изменения состояния задач и контролировать выполнение работ сотрудниками.

Для регистрации действий системы и диагностики ошибок реализована подсистема логирования на основе библиотеки zap. Все критические операции, ошибки и служебные сообщения автоматически сохраняются в журнал событий. На рисунке 2.4.15 представлен фрагмент программного кода настройки логирования приложения.



```

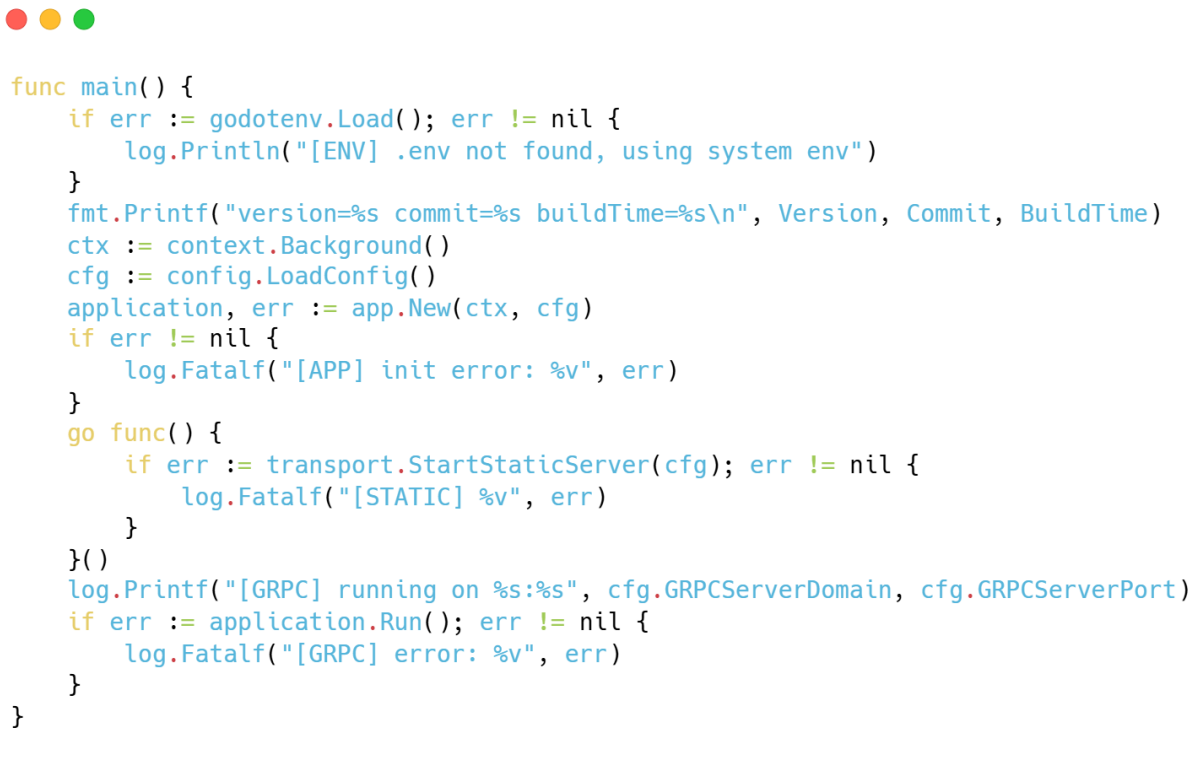
func NewLogger(env string) *zap.Logger {
    fileWriter := zapcore.AddSync(&lumberjack.Logger{
        Filename:   "logs/app.log",
        MaxSize:    10,
        MaxBackups: 5,
        MaxAge:     7,
        Compress:   true,
    })

    fileEncoder := zapcore.NewJSONEncoder(zap.NewProductionEncoderConfig())
    consoleConfig := zap.NewDevelopmentEncoderConfig()
    consoleConfig.EncodeLevel = CustomLevelEncoder
    consoleConfig.EncodeTime = zapcore.TimeEncoderOfLayout("15:04:05")
    consoleConfig.ConsoleSeparator = " "
    consoleEncoder := zapcore.NewConsoleEncoder(consoleConfig)
    level := zap.InfoLevel
    if env == "dev" {
        level = zap.DebugLevel
    }
    core := zapcore.NewTee(
        zapcore.NewCore(fileEncoder, fileWriter, zap.InfoLevel),
        zapcore.NewCore(consoleEncoder, zapcore.AddSync(os.Stdout), level),
    )
    return zap.New(core,
        zap.AddCaller(),
        zap.AddStacktrace(zap.ErrorLevel),
    )
}

```

Рисунок 2.4.15 - Фрагмент программного кода настройки логирования приложения

Во время запуска приложения выполняется загрузка конфигурации, подключение к базе данных PostgreSQL, регистрация сервисов, запуск подсистемы потоковой передачи данных и запуск gRPC-сервера. На рисунке 2.4.16 представлен фрагмент программного кода функции main.



```

func main() {
    if err := godotenv.Load(); err != nil {
        log.Println("[ENV] .env not found, using system env")
    }
    fmt.Printf("version=%s commit=%s buildTime=%s\n", Version, Commit, BuildTime)
    ctx := context.Background()
    cfg := config.LoadConfig()
    application, err := app.New(ctx, cfg)
    if err != nil {
        log.Fatalf("[APP] init error: %v", err)
    }
    go func() {
        if err := transport.StartStaticServer(cfg); err != nil {
            log.Fatalf("[STATIC] %v", err)
        }
    }()
    log.Printf("[GRPC] running on %s:%s", cfg.GRPCServerDomain, cfg.GRPCServerPort)
    if err := application.Run(); err != nil {
        log.Fatalf("[GRPC] error: %v", err)
    }
}

```

Рисунок 2.4.15 - Фрагмент программного кода функции main

Таким образом, разработанная серверная часть обеспечивает безопасную обработку данных пользователей, управление проектами и задачами, разграничение прав доступа и синхронизацию данных между участниками системы в режиме реального времени.

2.5 Тестирование информационной системы

После завершения разработки информационной системы “Atlas” было проведено функциональное тестирование программного продукта. Целью тестирования являлась проверка корректности работы реализованных функций и подтверждение соответствия системы требованиям технического задания.

В ходе тестирования проверялись основные возможности системы: регистрация и авторизация пользователей, управление рабочими пространствами, работа с участниками проектов, создание и редактирование

kanban-досок, управление задачами и файловыми вложениями, а также синхронизация данных между пользователями в режиме реального времени.

Тестирование выполнялось методом «черного ящика», при котором система рассматривалась с точки зрения конечного пользователя без анализа внутренней структуры программного кода.

В качестве тестовой среды использовались клиентское приложение Atlas, серверное приложение на языке Go, СУБД PostgreSQL и протокол gRPC для взаимодействия между компонентами системы.

Для проверки работоспособности программного продукта были разработаны и выполнены тестовые сценарии, охватывающие основные пользовательские операции. Результаты тестирования представлены в таблицах ниже.

Тест кейс №1 - Регистрация нового пользователя

Поле	Значение
Приоритет	High
Название	Регистрация нового пользователя
Предусловие	Пользователь не авторизован в системе. Email отсутствует в базе данных.
Тестовые данные	Email: ectobiologist@homestuck.com, Пароль: i_hate_cake, Имя: Джон, Фамилия: Эгберт.
Шаги	1. Запустить приложение “Atlas”. 2. Открыть форму регистрации. 3. Заполнить обязательные поля. 4. Нажать кнопку “Зарегистрироваться”.
Ожидаемый результат	Запрос регистрации успешно отправлен на сервер. Создается новая учетная запись пользователя. Сервер возвращает JWT токен и данные пользователя. Пользователь автоматически авторизуется в системе. Данные профиля сохраняются в локальной SQLite базе данных. Отображается главное окно приложения.
Статус прохождения Passed or Failed	Passed

Продолжение тест кейса №1

Поле	Значение
Комментарий	Проверена корректность взаимодействия клиента и gRPC сервера при регистрации пользователя.

Тест кейс №2 - Авторизация пользователя в системе

Поле	Значение
Приоритет	High
Название	Авторизация пользователя
Предусловие	В системе существует зарегистрированная учетная запись.
Тестовые данные	Email: ectobiologist@homestuck.com, Пароль: i_hate_cake.
Шаги	1. Запустить приложение “Atlas”. 2. Открыть форму авторизации. 3. Заполнить обязательные поля. 4. Нажать кнопку “Войти”.
Ожидаемый результат	Сервер успешно выполняет проверку учетных данных. Возвращается JWT токен доступа. Пользователь перенаправляется в основное окно приложения. Данные профиля сохраняются в локальной SQLite базе данных.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверено сохранение локальной сессии пользователя.

Тест кейс №3 - Создание рабочего пространства

Поле	Значение
Приоритет	High
Название	Создание рабочего пространства
Поле	Значение
Предусловие	Пользователь авторизован.
Тестовые данные	Название: Sburb Beta, Описание: Cool game for you and your friends.

Продолжение тест кейса №3

Поле	Значение
Шаги	1. Перейти в раздел Workspaces. 2. Нажать кнопку создания рабочего пространства. 3. Заполнить форму. 4. Подтвердить создание.
Ожидаемый результат	Новое рабочее пространство сохраняется в системе. Пользователь автоматически назначается владельцем. Рабочее пространство появляется в общем списке. Через gRPC Stream другим участникам отправляется событие WorkspaceCreated.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверена работа WorkspaceService.CreateWorkspace().

Тест кейс №4 - Приглашение пользователя в рабочее пространство

Поле	Значение
Приоритет	High
Название	Управление участниками рабочего пространства
Предусловие	Существует рабочее пространство.
Тестовые данные	Email приглашенного пользователя: turntechgodhead@homestuck.com, Роль: MEMBER
Шаги	1. Открыть настройки рабочего пространства. 2. Создать приглашение. 3. Выполнить вход под приглашенным пользователем. 4. Принять приглашение.
Ожидаемый результат	Приглашение создается успешно. Пользователь получает приглашение в разделе Workspace Invites. После подтверждения пользователь появляется в списке участников рабочего пространства.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверена работа WorkspaceInviteService.

Тест кейс №5 - Создание Kanban-доски

Поле	Значение
Приоритет	High
Название	Создание Kanban-доски
Предусловие	Существует рабочее пространство.
Тестовые данные	Название доски: Pesterchum
Шаги	1. Выбрать рабочее пространство. 2. Нажать кнопку создания доски. 3. Указать название. 4. Подтвердить создание.
Ожидаемый результат	Доска создается в выбранном рабочем пространстве. Отображается в списке досок. Сервер возвращает объект Board.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверена работа BoardService.CreateBoard().

Тест кейс №6 - Создание и редактирование колонок Kanban

Поле	Значение
Приоритет	High
Название	Создание и редактирование колонок Kanban
Предусловие	Создана Kanban-доска.
Тестовые данные	To Do, In Progress, Done
Шаги	1. Создать колонку To Do. 2. Создать колонку In Progress. 3. Создать колонку Done. 4. Изменить название одной из колонок.
Ожидаемый результат	Колонки создаются успешно. Изменения сохраняются. Интерфейс синхронизируется без перезагрузки страницы.
Статус прохождения Passed or Failed	Passed

Продолжение тест кейса №6

Поле	Значение
Комментарий	Проверены методы CreateColumn, UpdateColumn.

Тест кейс №7 - Создание и редактирование задач

Поле	Значение
Приоритет	High
Название	Создание и редактирование задач
Предусловие	Существует колонка Kanban-доски.
Поле	Значение
Тестовые данные	Название: Написать сообщение Дейву
Шаги	1. Создать задачу. 2. Открыть карточку задачи. 3. Изменить описание. 4. Изменить приоритет.
Ожидаемый результат	Карточка задачи создается. Все изменения сохраняются. Сервер возвращает обновленный объект Task.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверены CreateTask и UpdateTask.

Тест кейс №8 - Перемещение задач между колонками

Поле	Значение
Приоритет	High
Название	Перемещение задач между колонками
Предусловие	Существует несколько колонок и минимум одна задача.
Тестовые данные	Задача "Написать сообщение Дейву"
Шаги	1. Переместить задачу из колонки To Do в колонку In Progress. 2. Повторно переместить задачу в колонку Done.

Продолжение тест кейса №8

Поле	Значение
Ожидаемый результат	Поле column_id изменяется. Обновляется sort_key. Все подключенные клиенты получают событие TaskMoved через gRPC Stream.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверена работа Drag-and-Drop механизма.

Тест кейс №9 - Работа с вложениями

Поле	Значение
Приоритет	Medium
Название	Работа с вложениями
Предусловие	Существует задача.
Тестовые данные	karkat.png размером 500 КБ
Шаги	1. Открыть задачу. 2. Добавить файл. 3. Проверить отображение вложения. 4. Удалить вложение.
Ожидаемый результат	Вложение успешно загружается. Отображается в карточке задачи. После удаления исчезает из списка вложений.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверены AddAttachment и DeleteAttachment.

Тест кейс №10 - Проверка синхронизации данных через gRPC Stream

Поле	Значение
Приоритет	High
Название	Проверка синхронизации данных через gRPC Stream
Предусловие	Авторизованы два клиента Atlas под разными учетными записями.
Тестовые данные	Изменение названия рабочего пространства.

Продолжение тест кейса №10

Поле	Значение
Шаги	1. На первом клиенте изменить название рабочего пространства. 2. Дождаться обработки события сервером. 3. Проверить состояние второго клиента.
Поле	Значение
Ожидаемый результат	Второй клиент получает событие WorkspaceUpdated через потоковое соединение gRPC. Интерфейс обновляется автоматически без ручной перезагрузки.
Статус прохождения Passed or Failed	Passed
Комментарий	Проверена работа StreamEvents и системы событий реального времени.

По результатам тестирования установлено, что разработанная информационная система корректно реализует все функции, предусмотренные техническим заданием. Критических ошибок, препятствующих эксплуатации программного продукта, не выявлено.

Проверка подтвердила корректную работу механизмов регистрации и авторизации пользователей, управления рабочими пространствами, kanban-досками, задачами и вложениями, а также синхронизацию данных в режиме реального времени.

Все тестовые сценарии завершились успешно (Passed), что подтверждает соответствие системы требованиям технического задания и её готовность к практическому применению.

3 Экономическое обоснование

3.1 Определение затрат на проектирование

Для разработки программного продукта “Atlas”, предназначенного для организации командной работы, управления рабочими пространствами, kanban-досками и задачами, были определены основные затраты на проектирование и разработку системы.

В состав затрат входят:

- основная заработная плата разработчика;
- дополнительная заработная плата;
- страховые отчисления;
- расходы на программное обеспечение и серверную инфраструктуру;
- накладные расходы;
- затраты на машинное время.

Разработка системы выполнялась одним разработчиком в течение 15 рабочих дней при восьмичасовом рабочем дне. Общая продолжительность работ составила 120 часов.

График выполнения работ представлен в таблице 3.1.

Таблица 3.1 - График выполнения работ по разработке системы “Atlas”

№ п/п	Наименование работ	Исполнитель	Длительность в днях	Длительность в часах
1	Анализ требований и проектирование системы	Федин М.А.	3	24
2	Разработка серверной части системы	Федин М.А.	4	32
3	Разработка клиентской части системы	Федин М.А.	4	32

Продолжение таблицы 3.1

№ п/п	Наименование работ	Исполнитель	Длительность в днях	Длительность в часах
4	Реализация системы авторизации и рабочих пространств	Федин М.А.	2	16
5	Тестирование и отладка системы	Федин М.А.	2	16
Итого			15	120

Таким образом, на проектирование и разработку системы “Atlas” было затрачено 15 рабочих дней или 120 часов.

Для функционирования системы требуется использование доменного имени и серверного хостинга. Данные по расходам представлены в таблице 3.2.

Таблица 3.2 - Стоимость ресурсов, необходимых для работы системы

Наименование ресурса	Срок, мес.	Цена, руб.	Сумма, руб.
VPS сервер	12	900/мес.	10800
Доменное имя	12	250/год	250
Итого			11050

Общая стоимость ресурсов составляет

$$M = \text{VPS сервер} + \text{Доменное имя} \quad (3.1)$$

$$M = 10800 + 250 = 11050 \text{ руб.}$$

Расчет основной заработной платы осуществляется на основе дневной оплаты труда разработчика. Размер оплаты составляет 700 рублей.

Основная заработная плата рассчитывается по формуле:

$$Z_{осн} = T * C_{ч} \quad (3.2)$$

где T - количество отработанных часов;

Сч - стоимость одного часа работы.

Расчет основной заработной платы:

$$З_{осн} = 120 * 700 = 84000 \text{ руб.}$$

Дополнительная заработная плата принимается в размере 20% от основной заработной платы и рассчитывается по формуле:

$$З_{доп} = З_{осн} * 0,2 \quad (3.3)$$

$$З_{доп} = 84000 \times 0,2 = 16800 \text{ руб.}$$

Расчет фонда заработной платы представлен в таблице 3.3.

Таблица 3.3 - Расчет заработной платы

№ п/п	Наименование	Сумма, руб.
1	Основная заработная плата	84000
2	Дополнительная заработная плата	16800
Итого	Фонд заработной платы	100800

Отчисления на социальные нужды рассчитываются в размере 30% от фонда заработной платы:

$$З_{соц} = (З_{осн} + З_{доп}) * 0,3 \quad (3.4)$$

$$З_{соц} = 100800 \times 0,3 = 30240 \text{ руб.}$$

Накладные расходы принимаются в размере 20% от суммы основной и дополнительной заработной платы:

$$З_{н} = (З_{осн} + З_{доп}) * 0,2 \quad (3.5)$$

$$З_{н} = 100800 * 0,2 = 20160 \text{ руб.}$$

При разработке системы использовался персональный компьютер со средней мощностью энергопотребления 1,2 кВт. Среднее время работы за компьютером составляло 6 часов в день. Стоимость 1 кВт/ч электроэнергии составляет 9,1 рубля.

Суммарное энергопотребление за день рассчитывается по формуле:

$$P = 1,2 * 6 \quad (3.6)$$

$$P = 7,2 \text{ кВт/ч.}$$

Затраты на машинное время рассчитываются по формуле:

$$З_{\text{маш}} = P * Д_{\text{н}} * С1 \text{ кВт/ч} \quad (3.7)$$

$$З_{\text{маш}} = 7,2 * 15 * 9,1 = 982,8 \text{ руб.}$$

Итоговая смета затрат представлена в таблице 3.4.

Таблица 3.4 - Итоговая смета затрат на разработку системы “Atlas”

№ п/п	Наименование статьи расходов	Сумма, руб.
1	Основная заработная плата	84000
2	Дополнительная заработная плата	16800
3	Отчисления на социальные нужды	30240
4	Стоимость серверной инфраструктуры и домена	11050
5	Накладные расходы	20160
6	Затраты на машинное время	982,8
Итого		163232.8

Таким образом, общие затраты на проектирование и разработку системы “Atlas” составили 163232,8 рублей.

3.2 Расчет экономической эффективности

Система “Atlas” предназначена для автоматизации процессов управления задачами, рабочими пространствами и командным взаимодействием. Использование системы позволяет сократить время на распределение задач, коммуникацию между участниками проекта и контроль выполнения работ.

До внедрения системы значительная часть процессов выполнялась вручную с использованием мессенджеров, электронных таблиц и сторонних сервисов. В среднем команда из 6 сотрудников тратила около 12 часов в неделю на организационные процессы, включая постановку задач, согласование изменений и контроль выполнения работ.

После внедрения системы “Atlas” затраты времени на аналогичные операции сократились до 4 часов в неделю за счет централизованного управления задачами, встроенной системы рабочих пространств, kanban-досок и чатов задач.

Экономия времени составляет:

$$12 - 4 = 8 \text{ часов в неделю.}$$

Средняя стоимость одного часа работы сотрудника составляет 650 рублей.

Недельная экономия денежных средств:

$$8 * 650 = 5200 \text{ руб.}$$

Годовая экономия рассчитывается по формуле:

$$Д = 5200 * 52 \quad (3.8)$$

$$Д = 270400 \text{ руб.}$$

Экономический эффект рассчитывается как разница между годовой экономией и затратами на разработку системы:

$$Э = Д - Р \quad (3.9)$$

$$Э = 270400 - 163232,8 = 107167,2 \text{ руб.}$$

Срок окупаемости проекта рассчитывается по формуле:

$$O = P/D \quad (3.10)$$

$O = 163232,8 / 270400 \approx 0,6$ года, что составляет примерно 7 месяца.

Также разработанная система “Atlas” может использоваться не только внутри одной организации, но и распространяться как самостоятельный коммерческий программный продукт для других команд и компаний, занимающихся разработкой программного обеспечения и управлением проектами.

Система поддерживает работу с несколькими рабочими пространствами, управление участниками, kanban-досками и задачами, что позволяет адаптировать её под различные организации без необходимости существенной доработки функционала.

Возможная модель монетизации системы может включать:

- ежемесячную подписку для команд;
- размещение системы на серверной инфраструктуре заказчика;
- предоставление платных расширенных функций;
- техническую поддержку и сопровождение.

Средняя стоимость VPS-хостинга для размещения web-приложений подобного типа составляет от 300 до 1500 рублей в месяц в зависимости от конфигурации сервера и предполагаемой нагрузки системы.

При использовании модели подписки даже с минимальной стоимостью обслуживания в размере 3000-5000 рублей в месяц для одной организации разработанная система способна приносить стабильную прибыль и полностью окупить затраты на разработку за короткий период времени.

Таким образом, разработка и внедрение системы “Atlas” является экономически целесообразной, поскольку обеспечивает снижение временных затрат сотрудников, повышение эффективности взаимодействия внутри

команды, быструю окупаемость проекта и возможность дальнейшего коммерческого распространения системы как самостоятельного программного продукта.

Заключение

В ходе выполнения дипломного проекта был разработан программный продукт “Atlas”, предназначенный для организации командной работы, управления рабочими пространствами и контроля выполнения задач. В рамках работы была изучена предметная область, сформированы требования к системе, спроектирована архитектура приложения и реализовано программное решение, обеспечивающее централизованное управление проектной деятельностью пользователей.

Разработанная система построена на клиент-серверной архитектуре. Серверная часть реализована на языке программирования Go с использованием системы управления базами данных PostgreSQL и протокола gRPC с поддержкой потоковой передачи данных. Клиентская часть разработана с использованием технологий Wails и React, что позволило создать современное кроссплатформенное desktop-приложение для операционных систем Windows 11, Linux и macOS.

В результате разработки была реализована система аутентификации и авторизации пользователей с поддержкой регистрации по электронной почте и паролю, а также через внешних провайдеров. Реализованы механизмы управления учетной записью пользователя, включая изменение персональных данных, настройку параметров приложения, управление способами авторизации и пользовательскими предпочтениями интерфейса.

В программном продукте реализована система рабочих пространств, позволяющая создавать персональные и командные проекты, приглашать участников и управлять их ролями. Для обеспечения безопасности и контроля доступа предусмотрено разграничение прав пользователей на основе ролей владельца, администратора и участника.

Одной из ключевых возможностей системы является управление kanban-досками и задачами. Пользователям предоставлены средства создания досок, организации задач по колонкам, изменения их состояния и контроля процесса

выполнения работ. Дополнительно реализована возможность прикрепления файлов к задачам, что обеспечивает удобное хранение материалов, связанных с выполняемой деятельностью.

В процессе разработки была спроектирована структура базы данных, обеспечивающая целостность и согласованность хранимой информации. Реализованная серверная логика обеспечивает обработку запросов пользователей, управление рабочими пространствами, досками и задачами, контроль прав доступа, хранение данных и синхронизацию состояния между участниками проектов.

Разработанная система прошла тестирование, в ходе которого были проверены основные пользовательские сценарии, включая регистрацию и авторизацию пользователей, управление рабочими пространствами, взаимодействие с участниками проектов, работу с kanban-досками и задачами. Результаты тестирования подтвердили корректность функционирования реализованного программного обеспечения и соответствие системы поставленным требованиям.

Полученные результаты показывают, что разработанный программный продукт позволяет эффективно организовывать совместную работу пользователей, централизованно управлять задачами и обеспечивать удобное взаимодействие участников проектов в рамках единой информационной системы.

Таким образом, в ходе выполнения дипломного проекта были достигнуты поставленные цели: автоматизированы процессы управления задачами и рабочими процессами, обеспечена возможность совместной работы пользователей в рамках рабочих пространств, реализованы централизованное хранение и обработка данных, безопасная аутентификация и авторизация пользователей, разграничение прав доступа между участниками проектов, а также средства контроля выполнения задач. Разработанная архитектура программного продукта обеспечивает возможность дальнейшего расширения и развития системы. Программный продукт “Atlas” соответствует

предъявленным требованиям и может быть использован для организации и автоматизации процессов управления проектами и командной работы.

Библиография

1. Go Documentation [Электронный ресурс]. - Режим доступа: <https://go.dev/doc/> (дата обращения: 20.03.2026).
2. Effective Go [Электронный ресурс]. - Режим доступа: https://go.dev/doc/effective_go (дата обращения: 21.03.2026).
3. Go Modules Reference [Электронный ресурс]. - Режим доступа: <https://go.dev/ref/mod> (дата обращения: 22.03.2026).
4. gRPC Documentation [Электронный ресурс]. - Режим доступа: <https://grpc.io/docs/> (дата обращения: 23.03.2026).
5. gRPC Go Documentation [Электронный ресурс]. - Режим доступа: <https://grpc.io/docs/languages/go/> (дата обращения: 24.03.2026).
6. Protocol Buffers Documentation [Электронный ресурс]. - Режим доступа: <https://protobuf.dev/> (дата обращения: 25.03.2026).
7. PostgreSQL Documentation [Электронный ресурс]. - Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 26.03.2026).
8. pgx - PostgreSQL Driver for Go [Электронный ресурс]. - Режим доступа: <https://pkg.go.dev/github.com/jackc/pgx/v5> (дата обращения: 27.03.2026).
9. sqlc Documentation [Электронный ресурс]. - Режим доступа: <https://docs.sqlc.dev/> (дата обращения: 28.03.2026).
10. Goose - Database Migration Tool [Электронный ресурс]. - Режим доступа: <https://pressly.github.io/goose/> (дата обращения: 29.03.2026).
11. OAuth 2.0 Authorization Framework [Электронный ресурс]. - Режим доступа: <https://oauth.net/2/> (дата обращения: 30.03.2026).
12. Google Identity Platform Documentation [Электронный ресурс]. - Режим доступа: <https://developers.google.com/identity> (дата обращения: 31.03.2026).
13. GitHub OAuth Apps Documentation [Электронный ресурс]. - Режим доступа: <https://docs.github.com/en/apps/oauth-apps> (дата обращения: 01.04.2026).
14. JSON Web Tokens (JWT) Introduction [Электронный ресурс]. - Режим доступа: <https://jwt.io/introduction> (дата обращения: 02.04.2026).

- 15.bcrypt Package Documentation [Электронный ресурс]. - Режим доступа: <https://pkg.go.dev/golang.org/x/crypto/bcrypt> (дата обращения: 03.04.2026).
- 16.Wails Documentation [Электронный ресурс]. - Режим доступа: <https://wails.io/docs/> (дата обращения: 05.04.2026).
- 17.React Documentation [Электронный ресурс]. - Режим доступа: <https://react.dev/learn> (дата обращения: 06.04.2026).
- 18.React Router Documentation [Электронный ресурс]. - Режим доступа: <https://reactrouter.com/> (дата обращения: 07.04.2026).
- 19.Zap Documentation [Электронный ресурс]. – Режим доступа: <https://pkg.go.dev/go.uber.org/zap> (дата обращения: 08.04.2026).
- 20.react-i18next Documentation [Электронный ресурс]. - Режим доступа: <https://react.i18next.com/> (дата обращения: 09.04.2026).
- 21.JavaScript Documentation [Электронный ресурс]. - Режим доступа: <https://developer.mozilla.org/docs/Web/JavaScript> (дата обращения: 10.04.2026).
- 22.Zap: Blazing Fast, Structured, Leveled Logging in Go [Электронный ресурс]. – Режим доступа: <https://github.com/uber-go/zap> (дата обращения: 08.04.2026).
- 23.ESLint Documentation [Электронный ресурс]. - Режим доступа: <https://eslint.org/docs/latest/> (дата обращения: 12.04.2026).
- 24.Prettier Documentation [Электронный ресурс]. - Режим доступа: <https://prettier.io/docs/en/> (дата обращения: 13.04.2026).
- 25.React Hooks API Reference [Электронный ресурс]. - Режим доступа: <https://react.dev/reference/react> (дата обращения: 14.04.2026).
- 26.Kanban Method Overview [Электронный ресурс]. - Режим доступа: <https://kanbanize.com/kanban-resources/getting-started/what-is-kanban> (дата обращения: 15.04.2026).
- 27.Git Documentation [Электронный ресурс]. - Режим доступа: <https://git-scm.com/docs> (дата обращения: 16.04.2026).
- 28.GitHub Documentation [Электронный ресурс]. - Режим доступа: <https://docs.github.com/> (дата обращения: 17.04.2026).

- 29.REST vs gRPC Comparison [Электронный ресурс]. - Режим доступа:
<https://grpc.io/docs/what-is-grpc/> (дата обращения: 18.04.2026).
- 30.Desktop App Architecture Guide [Электронный ресурс]. - Режим доступа:
<https://wails.io/docs/howdoesitwork/> (дата обращения: 20.04.2026).

Приложение А

Техническое задание

1 Наименование программы

Наименование программы: “Atlas”

2 Краткая характеристика области применения

Приложение “Atlas” предназначено для организации и управления рабочими процессами в рамках командной разработки и совместной работы пользователей. Система позволяет создавать рабочие пространства (workspaces), управлять участниками, задачами и коммуникацией внутри команды в режиме реального времени.

3 Основания для разработки

Основанием для разработки является необходимость создания единой системы управления рабочими процессами и командным взаимодействием в рамках компании ООО “Интернет-Эксперт”.

Наименование темы разработки: Разработка веб приложения для мониторинга и управления рабочей деятельностью сотрудников ООО “Интернет-Эксперт”.

4 Назначение разработки

Приложение будет использоваться в компании ООО “Интернет-Эксперт” для организации внутренней командной работы, управления проектами и взаимодействия сотрудников в рамках рабочих пространств.

5 Функционально назначение

Программное обеспечение предназначено для:

- аутентификации и регистрации пользователей;
- создания и управления рабочими пространствами (workspaces);
- объединения пользователей в рабочие группы;
- обмена данными в режиме реального времени между клиентом и сервером.

6 Эксплуатационное назначение

Система предназначена для эксплуатации в корпоративной среде на персональных компьютерах пользователей под управлением Windows, macOS и Linux (за счёт использования Wails). Доступ к системе осуществляется через графический интерфейс приложения.

7 Требования к программе или программному изделию

7.1 Требования к функциональным характеристикам

7.1.1 Требования к составу выполняемых функций

После запуска программы пользователю отображается форма входа в учетную запись, которая требует обязательного ввода email и пароля. Также предусмотрена возможность авторизации через сторонние провайдеры в виде Google.

Ниже формы входа располагается переключатель, позволяющий перейти с формы авторизации на форму регистрации.

Форма регистрации требует обязательного ввода имени, фамилии, email и пароля. Дополнительно пользователь может указать отчество, однако данное поле является необязательным. Аналогично форме входа предусмотрена

возможность регистрации через внешние провайдеры Google. При создании пользователя создается начальное личное рабочее пространство (workspaces) пользователя.

После успешного выполнения входа или регистрации пользователь перенаправляется на dashboard, где осуществляется дальнейшая работа с системой.

В рамках системы пользователь получает доступ к рабочим пространствам (workspaces), при этом ему доступно создание нового workspace либо присоединение к уже существующему по приглашению. При создании workspace пользователь указывает его название, описание и изображение (аватар). После создания workspace автоматически становится доступным для участников системы, при этом создателю назначается роль “владелец”, обладающая полным набором прав управления данным пространством. Пользователи, присоединяющиеся к workspace, получают роль “администратор” либо “участник” в зависимости от уровня предоставленных прав доступа.

Интерфейс dashboard включает навигационные разделы “Пространства”, “Доски”, “Настройки” и “Профиль”.

Верхняя панель отображается только на страницах “Пространства” и “Доски” при условии выбранного проекта. Панель содержит изображение и название активного проекта, кнопку “Приглашения”, а также кнопку создания рабочего пространства, создания доски либо не отображает дополнительную кнопку в зависимости от текущего раздела системы.

Раздел “Пространства” предназначен для управления рабочими пространствами пользователя. Карточка рабочего пространства содержит информацию о названии, описании, роли текущего пользователя, типе пространства (“Персональный” или “Командный”), а также кнопки “Фокусировка” и “Управление”. При нажатии кнопки “Фокусировка” выбранное рабочее пространство становится активным, после чего в разделе “Доски” отображаются данные только данного пространства.

При открытии управления рабочим пространством всем пользователям доступна вкладка с общей информацией о проекте, включающей `uid` пространства, имя владельца, изображение, количество участников, количество отправленных приглашений, роль пользователя, дату создания и дату последнего обновления пространства. Также отображается кнопка “Покинуть проект”.

Во вкладке “Участники” отображается список всех участников рабочего пространства. Пользователям с ролью “admin” или “owner” доступна возможность изменения роли участников. Пользователю с ролью “owner” дополнительно доступна возможность передачи прав владельца другому участнику, после чего предыдущий владелец автоматически получает роль “admin”.

Вкладки “Приглашения” и “Редактирование” доступны только пользователям с ролью “admin” или “owner”. Во вкладке “Приглашения” пользователь может указать email приглашенного пользователя, выбрать роль и отправить приглашение на вступление в рабочее пространство. Данная вкладка не отображается для пространств с типом “Персональный”. После отправки приглашения отображается список пользователей, ожидающих подтверждения приглашения.

Во вкладке “Редактирование” доступно изменение изображения, названия и описания рабочего пространства. Также отображается кнопка удаления рабочего пространства, доступная только пользователю с ролью “owner”.

Раздел “Доски” предназначен для управления kanban-досками внутри рабочего пространства. Доска содержит название и может редактироваться только пользователями с ролью “admin” или “owner” в текущем рабочем пространстве. При открытии доски пользователь переходит к интерфейсу взаимодействия с kanban-доской.

Kanban-доска содержит колонки и находящиеся внутри них карточки задач. Создание колонок и карточек задач доступно только пользователям с

ролью “admin” или “owner”. Остальные пользователи могут перемещать карточки задач между колонками.

Карточка задачи содержит название, крайний срок выполнения и приоритет задачи. При открытии карточки открывается модальное окно с полной информацией о задаче. В окне также отображаются прикрепленные к задаче файлы. Возможность прикрепления и открепления файлов доступна всем участникам рабочего пространства.

Раздел “Настройки” предназначен для управления учетной записью пользователя и включает функцию выхода из системы. Помимо этого, в разделе реализовано управление параметрами приложения, включая смену темы интерфейса (светлая/тёмная), а также выбор языка интерфейса.

Раздел “Профиль” отображает основную информацию о пользователе, включая аватар, фамилию, имя, отчество и email. Также в данном разделе доступна функция редактирования учетных данных. При открытии функции редактирования отображается выезжающая боковая панель, позволяющая изменять персональную информацию пользователя, включая аватар, ФИО и email.

Изменение email возможно только при наличии установленного пароля учетной записи. Дополнительно пользователь может изменить пароль либо создать его, если регистрация была выполнена через стороннего провайдера. Также предусмотрена возможность привязки или отвязки внешних провайдеров авторизации Google.

Все критические действия в системе сопровождаются дополнительным подтверждением действия пользователя.

Приложение Б

SQL код таблиц базы данных

Код таблицы users

```
create table users (  
    user_id uuid primary key default gen_random_uuid(),  
    email text unique,  
    password_hash text,  
    first_name text not null,  
    last_name text not null,  
    middle_name text default "",  
    photo_url text default "",  
  
    created_at timestamptz default now(),  
    updated_at timestamptz default now()  
);
```

Код таблицы user_identities

```
create table user_identities (  
    user_identity_id uuid primary key default gen_random_uuid(),  
    user_id uuid references users(user_id) on delete cascade,  
    provider text not null,  
    provider_id text not null,  
  
    constraint unique_provider_id unique (provider, provider_id),  
    constraint unique_user_provider unique (user_id, provider)  
);
```

Код таблицы workspaces

```
create table workspaces (  
  workspace_id uuid primary key default gen_random_uuid(),  
  name text not null,  
  description text,  
  photo_url text,  
  is_personal boolean default false,  
  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);
```

Код таблицы workspace_invites

```
create table workspace_invites (  
  invite_id uuid primary key default gen_random_uuid(),  
  workspace_id uuid references workspaces(workspace_id) on delete cascade,  
  email text not null,  
  role workspace_role default 'member',  
  status workspace_invite_status default 'pending',  
  
  created_at timestamptz default now()  
);
```

Код таблицы workspace_members

```
create table workspace_members (  
    workspace_id uuid references workspaces(workspace_id) on delete cascade,  
    user_id uuid references users(user_id) on delete cascade,  
    role workspace_role default 'member',  
  
    joined_at timestamptz default now(),  
    primary key (workspace_id, user_id)  
);
```

Код таблицы boards

```
create table boards (  
    board_id uuid primary key default gen_random_uuid(),  
    workspace_id uuid references workspaces(workspace_id) on delete cascade,  
    name text not null,  
  
    created_at timestamptz default now(),  
    updated_at timestamptz default now()  
);
```

Код таблицы board_columns

```
create table board_columns (  
    column_id uuid primary key default gen_random_uuid(),  
    board_id uuid references boards(board_id) on delete cascade,  
    name text not null,  
    sort_key text not null,  
  
    created_at timestamptz default now()  
);
```

Код таблицы tasks

```
create table tasks (  
  task_id uuid primary key default gen_random_uuid(),  
  workspace_id uuid references workspaces(workspace_id) on delete cascade,  
  board_id uuid references boards(board_id) on delete cascade,  
  column_id uuid references board_columns(column_id) on delete set null,  
  parent_task_id uuid references tasks(task_id) on delete cascade,  
  
  title text not null,  
  description text,  
  
  priority task_priority default 'medium',  
  
  reporter_id uuid references users(user_id),  
  assignee_id uuid references users(user_id),  
  
  due_date timestamptz null,  
  sort_key text not null,  
  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);
```

Код таблицы task_attachments

```
create table task_attachments (  
    attachment_id uuid primary key default gen_random_uuid(),  
    task_id uuid references tasks(task_id) on delete cascade,  
    user_id uuid references users(user_id),  
  
    file_name text,  
    file_url text,  
    file_type text,  
    file_size bigint,  
  
    created_at timestamptz default now()  
);
```